

# Ghost Mechanism: An Analytical Model of Abrupt Learning in Recurrent Networks

Fatih Dinc<sup>1,2,3,4,\*</sup> Ege Cirakman<sup>3,\*</sup> Bariscan Kurtkaya<sup>3,5,\*</sup> Mert Yuksekgonul<sup>6</sup>  
 Yiqi Jiang<sup>3</sup> Mark J. Schnitzer<sup>3,7,8,†</sup> and Hidenori Tanaka<sup>4,9,†</sup>

<sup>1</sup>Kavli Institute for Theoretical Physics, *University of California, Santa Barbara, California 93106, USA*

<sup>2</sup>Geometric Intelligence Laboratory, *University of California, Santa Barbara, Santa Barbara, California 93106, USA*

<sup>3</sup>CNC Program, *Stanford University, Stanford, California 94305, USA*

<sup>4</sup>Physics of Artificial Intelligence Group, *NTT Research Inc., Sunnyvale, California 94085, USA*

<sup>5</sup>KUIS AI, Department of Computer Engineering, *Koc University, Istanbul, Turkey*

<sup>6</sup>Department of Computer Science, *Stanford University, Stanford, California 94305, USA*

<sup>7</sup>James H. Clark Center for Biomedical Engineering and Sciences, *Stanford University, Stanford, California 94305, USA*

<sup>8</sup>Howard Hughes Medical Institute, *Stanford University, Stanford, California 94305, USA*

<sup>9</sup>CBS-NTT Program in Physics of Intelligence, *Harvard University, Cambridge, Massachusetts 02138, USA*

 (Received 9 January 2025; revised 17 May 2025; accepted 8 April 2026; published 23 June 2026)

Abrupt learning, long performance plateaus followed by rapid convergence, is a common phenomenon in recurrent neural networks (RNNs) trained on working-memory tasks. In such cases, the networks develop transient slow regions in state space that extend the effective timescales of computation. However, the mechanisms driving sudden performance improvements and their causal role remain unclear, largely because we lack an analytical dynamical-systems framework. To address this gap, we introduce the ghost mechanism, a general process by which finite-dimensional continuous-time dynamical systems exhibit transient slowdown near the remnant of a saddle-node bifurcation. By reducing the high-dimensional dynamics near ghost points, we derive a one-dimensional canonical form that analytically captures learning as a process controlled by a single scale parameter. Using this model, we study a form of abrupt learning emerging from ghost points and identify a critical learning rate that scales as an inverse power law with the timescale of the learned computation. Beyond this rate, learning collapses through two interacting modes: (i) vanishing gradients and (ii) oscillatory gradients near minima. These features can lock the system into high confidence but incorrect predictions when parameter updates trigger a no-learning zone, a region of parameter space where gradients vanish. We validate these predictions in low-rank RNNs, where ghost points precede abrupt transitions and further demonstrate their generality in full-rank RNNs trained on canonical working-memory tasks. Our theory offers two approaches to address these learning difficulties: Increasing trainable ranks stabilizes learning trajectories, while reducing output confidence mitigates entrapment in no-learning zones. Overall, the ghost mechanism reveals how the computational demands of a task constrain the optimization landscape, demonstrating that well-known learning difficulties in RNNs partly arise from the dynamical systems they must learn to implement.

DOI: [10.1103/mjcl-lb4x](https://doi.org/10.1103/mjcl-lb4x)

Subject Areas: Complex Systems,  
Computational Physics

## I. INTRODUCTION

A hallmark of natural and artificial neural networks is the emergence of new capabilities, where computational

and behavioral algorithms are organically learned from experience [1–8]. Empirically, such an emergence often manifests itself as abrupt transitions in learning dynamics, sudden drops in loss functions that mark the acquisition of new computational abilities [9–19]. Such abrupt transitions are frequently accompanied by instabilities, including exploding or vanishing gradients and sharp shifts in network behavior [20–22]. While numerous studies have documented slow learning followed by sudden performance jumps [23,24], the underlying mechanisms driving these emergent transitions, even within specific domains of networks or classes of tasks, remain poorly understood. In

\*Principal author.

†These authors contributed equally to this work.

Published by the American Physical Society under the terms of the [Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/) license. Further distribution of this work must maintain attribution to the author(s) and the published article's title, journal citation, and DOI.

this work, we perform one such mechanistic study for recurrent neural networks (RNNs) trained to perform working-memory tasks.

One key challenge in studying the causal mechanisms subserving abrupt learning is that most approaches remain tied to numerical analyses [10,25], particular network architectures [9,10,26], or simplified linear models [27,28], making it difficult to identify general principles. Yet, at least intuitively, one instance of universality is expected if one focuses on temporal problems solved by dynamical systems: These systems must somehow reorganize their internal dynamics to suddenly perform new computations. This observation suggests that, rather than cataloging emergent behaviors across different systems, we might instead search for universal mechanisms, i.e., motifs in solutions that different architectures tend to converge to.

To understand what these universal mechanisms might be, it is instructive to examine how different research communities have approached abrupt learning. In modern deep networks, abrupt drops in loss with steplike gains in capability are often described as emergent phenomena, especially at large model, data, and compute scales [15–17,19]. While these studies provide extensive documentation of when and where such transitions occur, most accounts remain tied to specific architectural assumptions [17] and rely on empirical evidence without identifying the underlying dynamical motifs [29]. This empirical focus, though valuable for cataloging the phenomenon, does not reveal whether different systems achieve abrupt learning through shared mechanisms.

By contrast, dynamical-systems theory offers tools to identify such universal mechanisms, though traditionally studied mostly in the context of few-dimensional dynamical systems [30]. Then, sharp performance shifts do not need to depend on the particular architecture being trained, rather on the general properties of (low-dimensional) dynamical systems approximated by these architectures [31], whereas properties of the learned dynamical systems can be analyzed using established mathematical frameworks [32]. For instance, wisdom from classical bifurcation theory [33] would suggest that abrupt learning can occur when training drives the system across qualitative thresholds in its flow [9], suggesting that bifurcations might be one such universal motif underlying sudden behavioral changes [9,30]. This mathematical perspective shifts focus from architecture-specific details to dynamical structures that are learned during training. In fact, these agnostic approaches have proven particularly valuable in computational neuroscience, where analyses of RNNs have identified bifurcations as key drivers of both learning instabilities [20,21] and the emergence of new computational capabilities [9,11]. However, since both the parameters and the state variables are high dimensional in the trained architectures, the precise causal mechanisms underlying such phenomena often remain poorly understood.

In this work, we identify one such universal mechanism underlying abrupt learning in dynamical-system models trained to perform working-memory tasks: The formation of “ghost points,” i.e., transient bottlenecks in the dynamics of state variables that emerge near the remnants of saddle-node bifurcations [30]. Our analysis reveals how ghosts emerge through learning, why they cause training instabilities, and how these instabilities can be mitigated. Beyond providing one analytically tractable model of abrupt learning, this work discusses testable predictions for biological neural networks, practical strategies for training recurrent networks effectively, and potential insights for learning in deep neural networks.

We develop this analysis systematically as follows. In Sec. II, we introduce an analytically tractable one-dimensional canonical model of the ghost mechanism, trained on a delayed-activation task. This model captures a form of abrupt learning as a transient slowdown in the dynamics of trained systems. Next, in Sec. III, we show that the ghost mechanism reproduces the behavior of rank-one and rank-two RNNs trained on working-memory tasks, including their training instabilities. Section IV extends these insights to full-rank RNNs, where we identify two strategies to stabilize learning: (i) reducing output confidence and (ii) increasing the number of trainable ranks. In our Discussion (Sec. V), we situate our results within the broader literature on machine learning and computational neuroscience, explain how the uncovered failure modes (no-learning zones) differ from vanishing gradient problems in feedforward or recurrent neural networks that arise from saturated neuronal activation functions [20,28,40], and discuss their implications for autoregressive deep-learning architectures that must learn long-term dependencies without explicit mechanisms such as attention [41]. Finally, we conclude that addressing these constraints on training dynamical systems may shed light on learning in biological neural networks.

## II. GHOST MECHANISM

In this section, we introduce the ghost mechanism and motivate its broader relevance for dynamical systems. To start with, we derive a mathematical reduction of ghosts from high-dimensional systems to a one-dimensional system that will serve as our canonical model. Using this model, we conduct analytical derivations of the learning dynamics and extract insights regarding the abrupt learning of ghost points.

Throughout this work, we refer to scalars with lowercase letters such as  $a \in \mathbb{R}$ . We use lowercase letters with bold fonts to denote the vectors such as  $\mathbf{a} \in \mathbb{R}^N$ , a vector of  $N$  element. Then,  $a_i$  refers to the  $i$ th element of the vector. Uppercase letters with bold fonts are reserved for matrices such as  $\mathbf{A} \in \mathbb{R}^{N \times N}$ , a matrix of size  $N$  by  $N$ . Then,  $A_{ij}$  refers to the  $(i, j)$ th element of the matrix.

### A. Ghosts as temporal bottlenecks in high-dimensional dynamics

To introduce and motivate the generality of the ghost mechanism, we first define a general finite-dimensional continuous-time dynamical system

$$\dot{\mathbf{x}}(t) = \mathbf{f}[\mathbf{x}(t)], \quad \mathbf{x} \in \mathbb{R}^N, \quad (1)$$

where we refer to  $\mathbf{f}: \mathbb{R}^N \rightarrow \mathbb{R}^N$  as the vector field and  $\mathbf{x}$  as the state variables. We consider the behavior of this system in the neighborhood of local minima of the (kinetic) energy function,

$$s(\mathbf{x}) = \frac{1}{2} \|\mathbf{f}(\mathbf{x})\|_2^2. \quad (2)$$

We refer to a state variable configuration, in which this function attains a local minimum, as a local energy (and speed) minimum  $\mathbf{x}^*$ . We use a colloquial umbrella term “dynamical bottleneck” to refer to a compact region around such minima. One form of local minima is called a fixed point, for which  $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ . Not all local minima of energy functions have this property, and the main goal of this subsection is to precisely define these different types of local minima, i.e., slow points and ghost points, for general high-dimensional dynamical systems [42]. In the next subsection, using these definitions, we reduce the high-dimensional dynamics to a one-dimensional variable that captures the effective behavior of the system in the neighborhood of a ghost point  $\mathbf{x}^*$ . Such reductions are classical in dynamical-systems theory, where slow channels in high dimensions are often captured by one-dimensional normal forms (e.g., center manifold theory and saddle-node bifurcations [30,32,43]). This reduction will eventually allow an analytical study of learning dynamics as ghosts are formed in dynamical systems.

*Definition.* Slow point: We define a slow point at a location  $\mathbf{x}^*$  as one that satisfies the following conditions:

$$\nabla s(\mathbf{x}^*) = \mathbf{0}, \quad \nabla^2 s(\mathbf{x}^*) \succ \mathbf{0}, \quad s(\mathbf{x}^*) \neq 0. \quad (3)$$

Here, we define  $\nabla s(\mathbf{x}^*) \in \mathbb{R}^N$  as the gradient with respect to  $\mathbf{x}$  and  $\nabla^2 s(\mathbf{x}^*) \in \mathbb{R}^{N \times N}$  as the Hessian matrix. The first condition ensures that the energy function has an extremum at  $\mathbf{x}^*$ , the second makes it a local minimum [44], and the third guarantees that the flow does not vanish at  $\mathbf{x}^*$ , i.e., creating a distinction between fixed and slow points.

Following this definition, differentiating  $s(\mathbf{x})$  gives

$$\nabla s(\mathbf{x}) = \mathbf{J}(\mathbf{x})^T \mathbf{f}(\mathbf{x}), \quad (4)$$

where  $J_{ij} = [(\partial f_i)/(\partial x_j)]$  is the Jacobian of  $\mathbf{f}(\mathbf{x})$ . The extremum condition  $\nabla s(\mathbf{x}^*) = \mathbf{0}$  is equivalent to

$$\mathbf{J}(\mathbf{x}^*)^T \mathbf{f}(\mathbf{x}^*) = \mathbf{0} \Leftrightarrow \mathbf{f}(\mathbf{x}^*)^T \mathbf{J}(\mathbf{x}^*) = \mathbf{0}^T. \quad (5)$$

This orthogonality relation, in conjunction with the third condition that  $\mathbf{f}(\mathbf{x}^*) \neq \mathbf{0}$ , suggests that the left eigenvectors of the Jacobian at  $\mathbf{x}^*$  include  $\mathbf{f}(\mathbf{x}^*)$  with a corresponding eigenvalue of zero.

With the Jacobian in hand, we can now define the “ghost” as a special case of a slow point. Classically, a one-dimensional ghost is a remnant of a saddle-node bifurcation that slows down nearby trajectories [30] (the term is also used colloquially in higher-dimensional settings [45]). To generalize this definition to higher dimensions, we first define what it means for a slow point to be transverse stable:

*Assumption.* Transverse stability at a slow point: Let  $\mathbf{x}^*$  be a slow point and set  $\mathbf{v} := \mathbf{f}(\mathbf{x}^*)/\|\mathbf{f}(\mathbf{x}^*)\|_2$ . By Eq. (5),  $\mathbf{v}^T \mathbf{J}(\mathbf{x}^*) = \mathbf{0}^T$ . We assume that, for any  $\mathbf{w} \in \mathbf{v}^\perp \setminus \{\mathbf{0}\}$ , defined as collections of  $\mathbf{w} \neq \mathbf{0}$  satisfying  $\mathbf{v}^T \mathbf{w} = 0$ , the following statement is true:

$$\mathbf{w} \in \mathbf{v}^\perp \setminus \{\mathbf{0}\}, \quad \mathbf{w}^T \mathbf{J}(\mathbf{x}^*) = \lambda \mathbf{w}^T \Rightarrow \text{Re}[\lambda] < 0. \quad (6)$$

Here  $\text{Re}[\lambda]$  refers to the real part of the eigenvalue  $\lambda$ .

Notably, slow points in higher dimensions, including those created after saddle-node bifurcations, can have mixed stability in the transverse directions. However, when all transverse directions are stable, the resulting slow point acts as a funnel that rapidly attracts nearby trajectories onto a one-dimensional dynamical bottleneck. As we show in this work, this geometric structure proves particularly valuable for analyzing RNN dynamics. We therefore define ghosts as slow points satisfying this additional constraint:

*Definition.* Ghost: A ghost is a slow point  $\mathbf{x}^*$  that satisfies the transverse stability assumption.

As we show next, under the linearized dynamics near a ghost  $\mathbf{x}^*$ , the components transverse to  $\mathbf{f}(\mathbf{x}^*)$  decay exponentially so that local trajectories are rapidly confined to the one-dimensional dynamical bottleneck.

### B. Reduction of ghost dynamics to a one-dimensional canonical form

With the definition of a ghost point, we have all the pieces ready to perform the reduction to a one-dimensional system. Recall  $\mathbf{v} = \mathbf{f}(\mathbf{x}^*)/\|\mathbf{f}(\mathbf{x}^*)\|_2$  such that  $\mathbf{v}^T \mathbf{v} = 1$  and let  $\mathbf{z}(t) = \mathbf{x}(t) - \mathbf{x}^*$  denote the displacement for some  $\mathbf{x}(t)$  nearby  $\mathbf{x}^*$ . We perform Taylor expansions of the vector field for small  $\mathbf{z}(t)$ , which we can divide into two components as

$$\mathbf{z}(t) = \kappa(t) \mathbf{v} + \mathbf{z}_\perp(t), \quad \mathbf{v}^T \mathbf{z}_\perp(t) = 0. \quad (7)$$

Here,  $\kappa(t) = \mathbf{v}^T \mathbf{z}(t)$  is a scalar parametrizing the evolution along the slow mode when  $\mathbf{x}(0) = \mathbf{x}^*$ . Then, to the first-order corrections near the ghost point, the flow follows:

$$\dot{\mathbf{z}}(t) = \mathbf{f}[\mathbf{x}^* + \mathbf{z}(t)] \approx \mathbf{f}(\mathbf{x}^*) + \mathbf{J}(\mathbf{x}^*) \mathbf{z}(t). \quad (8)$$

Inserting  $\mathbf{z}(t)$  into this equation and projecting down to the perpendicular component, we arrive at the equation

$$\begin{aligned}\dot{\mathbf{z}}_{\perp}(t) &= \mathbf{J}(\mathbf{x}^*)\mathbf{z}_{\perp}(t) + \kappa(t)\mathbf{J}(\mathbf{x}^*)\mathbf{v}, \\ &= \mathbf{J}(\mathbf{x}^*)\mathbf{z}_{\perp}(t) + O(\kappa).\end{aligned}\quad (9)$$

For  $\kappa(0) = 0$  and  $\mathbf{z}_{\perp}(0) = \mathbf{0}$ , we have that  $\dot{\mathbf{z}}_{\perp}(0) = \mathbf{0}$  and, consequently,  $\mathbf{z}(t \approx 0) \approx \mathbf{v}\kappa(t)$  with  $\dot{\kappa}(t) = \mathbf{v}^T \mathbf{f}[\mathbf{x}^* + \mathbf{z}(t)] \approx \|\mathbf{f}(\mathbf{x}^*)\|_2 \neq 0$ . For  $t > 0$ , as long as  $\mathbf{z}(t)$  remains small, hence Taylor approximation is valid, each element of  $\mathbf{z}_{\perp}(t)$  decays rapidly to a value at approximately  $O(\kappa)$  since the spectrum of  $\mathbf{J}(\mathbf{x}^*)$  has only negative real parts by the transverse stability assumption. Since we are not interested in the exact flow direction near the ghost, but in the scalar variable  $\kappa(t) = \mathbf{v}^T \mathbf{z}(t)$ , we denote the limiting behavior as  $\mathbf{z} = \mathbf{z}_{\perp} + \kappa \mathbf{v} \approx \tilde{\mathbf{v}}\kappa$  for some  $\tilde{\mathbf{v}}$ . This allows us to analyze the displacement along  $\mathbf{v}$  quantitatively by keeping the second-order terms of the Taylor expansion,

$$\begin{aligned}\dot{\kappa}(t) &= \mathbf{v}^T \dot{\mathbf{x}}(t) = \mathbf{v}^T \mathbf{f}(\mathbf{x}^* + \mathbf{z}), \\ &= \|\mathbf{f}(\mathbf{x}^*)\|_2 + \frac{\kappa^2(t)}{2} \mathbf{v}^T \mathbf{h}[\tilde{\mathbf{v}}, \tilde{\mathbf{v}}] + O(\kappa^3).\end{aligned}\quad (10)$$

Here,  $\mathbf{h}[\mathbf{z}, \mathbf{z}]$  is the vector whose  $i$ th entry is  $\mathbf{z}^T \mathbf{H}_i(\mathbf{x}^*) \mathbf{z}$ , with  $\mathbf{H}_i(\mathbf{x}^*) = \nabla^2 f_i(\mathbf{x}^*) \in \mathbb{R}^{N \times N}$ . This equation has a constant term  $\gamma = \|\mathbf{f}(\mathbf{x}^*)\|_2 > 0$  and a quadratic term with coefficient  $\beta = \frac{1}{2} \mathbf{v}^T \mathbf{h}[\tilde{\mathbf{v}}, \tilde{\mathbf{v}}]$  that is assumed to be nonzero. However, there is no linear term, since  $\mathbf{v}^T \mathbf{J}(\mathbf{x}^*) = \mathbf{0}^T$  by Eq. (5), which leads to the reduced dynamics on the slow-mode  $\dot{\kappa} = \gamma + \beta\kappa^2 + O(\kappa^3)$ . Rescaling  $\kappa \rightarrow \kappa/\beta$  and defining  $r = \gamma\beta$ , we arrive at the equation

$$\dot{\kappa} = \kappa^2 + r + O(\kappa^3). \quad (11)$$

This simple one-dimensional dynamical system, effectively characterized by a single scale parameter  $r$  when higher-order terms are ignored, describes the universal behavior of the state variables  $\kappa(t)$  near the ghost point  $\kappa^* = 0$ . Below, ignoring the  $O(\kappa^3)$  corrections, we study the learning dynamics governing the scale parameter  $r$  in order to achieve a target passage time of the system variable  $\kappa(t)$  through the bottleneck  $\kappa = 0$ . As a side note, since our interest is in speed and not energy, and energy  $[\frac{1}{2} \|\mathbf{f}(\mathbf{x})\|_2^2]$  and speed  $[\|\mathbf{f}(\mathbf{x})\|_2]$  functions both share the same minima, we will refer to  $\mathbf{x}^*$  as speed minima for the rest of this work.

### C. Setting up the canonical model and the delayed-activation task

The dynamics near a ghost, as derived in Eq. (11), holds only for small  $\kappa$ , as higher-order terms can dominate for large  $\kappa \gg 1$ . However, since our goal is to study the time delay caused by very small parameter values  $0 < r \ll 1$ , we can focus instead on a simplified canonical model [Fig. 1(a)],

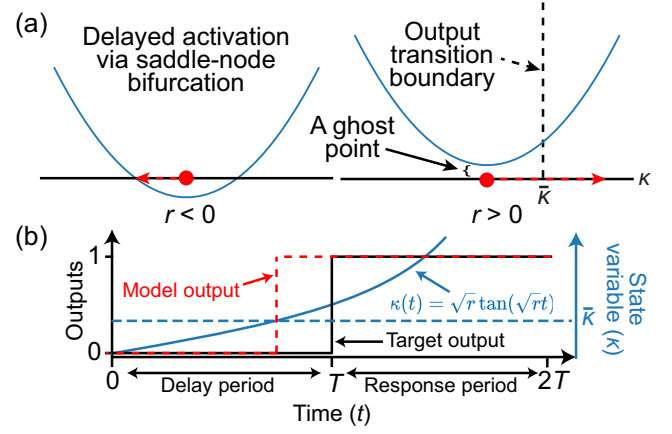


FIG. 1. Visualization of the canonical model and the delayed-activation task. (a) In our canonical model, the state variable is initialized at  $\kappa(0) = 0$  (red dot). Left: for  $r < 0$ , the system evolves toward a fixed point at  $-\sqrt{-r}$ . Right: when the system undergoes saddle-node bifurcation as  $r$  crosses zero,  $\kappa(t)$  diverges (red arrow). A predefined threshold  $\bar{\kappa}$  partitions the model output into two states, with  $\kappa > \bar{\kappa}$  corresponding to an activated response. A ghost point refers to the remnant of a saddle-node bifurcation at the origin, which is a fixed-point-like structure emerging from the annihilation of two fixed points. The trajectories considerably slow down and pass the ghost point in times  $\propto 1/\sqrt{r}$  [30]. (b) A schematic illustration of an example model output for a given  $r$ . The DA task requires the model to suppress its output during the delay period and initiate a transition during the response period. In our canonical model, this is achieved by ensuring that the state variable  $\kappa(t)$  remains below a predefined threshold  $\bar{\kappa}$  throughout the delay period and crosses it thereafter. In this example, reducing  $r$  would be needed to slow down the trajectory,  $\kappa(t) = \sqrt{r} \tan(\sqrt{r}t)$ , extending the time spent below threshold  $\kappa(t) \leq \bar{\kappa}$ .

$$\dot{\kappa}(t) = r + \kappa^2(t), \quad (12)$$

where higher-order corrections have been dropped. Here  $\kappa \in \mathbb{R}$  is the scalar state variable and  $r \in \mathbb{R}$  is now learnable. Since Eq. (12) has a simple quadratic form, assuming that the initial condition  $\kappa(0) = 0$ , and for  $r > 0$ , the solution can be written as

$$\kappa(t) = \begin{cases} \sqrt{r} \tan(\sqrt{r}t), & t \leq t^*, \\ \infty, & \text{otherwise,} \end{cases} \quad (13)$$

where  $t^* = [\pi/(2\sqrt{r})]$  is the finite escape time at which  $\kappa(t)$  diverges. This finite-time divergence is not physically meaningful, nor theoretically interesting to our cause, since the higher-order terms in Eq. (11) can alter the global landscape, for example, by introducing an additional fixed point at  $\kappa \gg 0$ .

In this work, we focus on the behavior of  $\kappa(t)$  near the origin for small  $r \ll 1$ . In this parameter regime, the trajectory remains confined near  $\kappa \approx 0$  for a duration of  $t \sim O(r^{-0.5})$  before escaping the bottleneck [30,46,47].

During this time, since  $\kappa[t \sim O(r^{-0.5})] \sim O(\sqrt{r}) \ll 1$ , the quadratic term  $[\kappa^2 \sim O(r)]$  contributes comparably to the constant term ( $r$ ), while higher-order corrections such as  $O(\kappa^3) \sim O(r\sqrt{r})$  remain negligible. Once the trajectory escapes this neighborhood and  $\kappa(t) \sim O(1)$ , higher-order corrections in Eq. (11) can have non-negligible effects on the subsequent dynamics. Thus, since our goal is to determine when the system escapes the bottleneck, i.e., the time at which  $\kappa(t)$  exceeds a user-defined threshold  $\bar{\kappa}$ , Eq. (12) captures the essential ghost dynamics and provides the appropriate canonical form that we will use in our derivations below.

Returning to the qualitative study of the canonical model, we identify the local fixed-point structure: There are two fixed points for  $r < 0$  and none for  $r > 0$ . The change in the number of fixed points at  $r = 0$  signals a saddle-node bifurcation, which is well characterized through bifurcation diagrams [30]. The slowdown of dynamics near ghost points for  $r > 0$  is textbook knowledge [30], whose computational properties continue to be studied in contemporary work [47]. To study learning dynamics, however, we need a task suited to the innate capabilities of ghosts. Fortunately, ghosts can function as delayed switches [46], in which a state of 0 can be assigned to  $\kappa \ll O(\sqrt{r})$  and a state of 1 for  $\kappa \gg O(\sqrt{r})$ . As we just discussed, when  $\kappa$  is initialized near the ghost, this construction achieves a delay of  $O(1/\sqrt{r})$  before switching between two states. In line with this observation, we train the canonical dynamical system to perform a delayed-activation (DA) task [Fig. 1(b)]. In this task, the state variable is initialized with  $\kappa(0) = 0$ . We define the model output as  $\hat{o}(\kappa) = \sigma[c(\kappa - \bar{\kappa})]$ , where  $\bar{\kappa} > 0$  and  $c > 0$  are predefined constants, and  $\sigma$  is the sigmoid function. In the limit  $c \rightarrow \infty$ , this output confidently takes on binary values, either 0 or 1; hence, we call  $c$  the confidence parameter.

The objective of the task is for the model's output  $[\hat{o}[\kappa(t)]]$  to match the target output  $[o(t)]$  of 0 until some predefined time point  $t < T$  and transition to 1 for  $T < t < 2T$ . Since the only learnable parameter is  $r$ , the learning process should pick an optimal value  $r^*$  to ensure this condition. Intuitively, one expects the solution to scale as  $r^* \sim O(T^{-2})$ , so that  $\kappa(t)$  escapes the ghost near  $\kappa^* = 0$  around  $t = T$ , coinciding with the required switch in output. It is less clear, however, whether minimizing a practical loss function can indeed recover this solution, which is what we investigate next.

#### D. Analytical derivations of the loss function and its gradient with the canonical model

We train the canonical model, i.e., update  $r$  in Eq. (12), to perform the DA task by minimizing a loss function,

$$\mathcal{L}(r) = \int_0^{2T} (\hat{o}[\kappa(t)] - o(t))^2 dt. \quad (14)$$

For analytical convenience, we assume the limit  $c \rightarrow \infty$  (referred to as the analytical limit alongside with  $\bar{\kappa} \rightarrow \infty$ ) such that the model output takes the form

$$\hat{o}[\kappa(t)] = \Theta[\kappa(t) - \bar{\kappa}], \quad (15)$$

for some predefined  $\bar{\kappa} > 0$ , where  $\Theta(\cdot)$  is the Heaviside function with  $\Theta(\kappa) = 1$  if  $\kappa > 0$  and 0 otherwise. Using this output, for  $r < 0$ , the loss function in Eq. (14) is trivially  $T$ , since the model output is always 0. Therefore, we will focus on the case with  $r > 0$ .

In the limit  $\bar{\kappa} \rightarrow \infty$ , we will simply assume that the network output is 0 before the escape, i.e.,  $t \leq t^*$ , and 1 after the escape, i.e.,  $t \geq t^*$ . Using this, we can explicitly compute the loss function,

$$\begin{aligned} \mathcal{L}(r) &= \int_0^{2T} dt \{ \hat{o}[\kappa(t)] - o(t) \}^2, \\ &= \int_0^{t^*} dt \{ \hat{o}[\kappa(t)] \}^2 + \int_{t^*}^{2T} dt \{ \hat{o}[\kappa(t)] - 1 \}^2. \end{aligned} \quad (16)$$

There are three important regimes here:

- (i)  $t^* \leq T$ , when the output turns on too early,
- (ii)  $T \leq t^* \leq 2T$ , when the output turns on late, and
- (iii)  $2T \leq t^*$ , when the output never turns on during the task.

Let us start with the first case,  $t^* \leq T$ ,

$$\mathcal{L}(r) = \int_{t^*}^T dt = T - t^* = T - \frac{\pi}{2\sqrt{r}}, \quad \text{for } r \geq \frac{\pi^2}{4T^2}. \quad (17)$$

Next, we consider the second case,  $T \leq t^* \leq 2T$ ,

$$\mathcal{L}(r) = \int_T^{t^*} dt = \frac{\pi}{2\sqrt{r}} - T, \quad \text{for } \frac{\pi^2}{16T^2} \leq r \leq \frac{\pi^2}{4T^2}. \quad (18)$$

Finally, we consider the case  $t^* \geq 2T$ ,

$$\mathcal{L}(r) = T \quad \text{for } r \leq \frac{\pi^2}{16T^2}. \quad (19)$$

When the output never turns on (i.e.,  $t^* \geq 2T$ ), the loss function is constant. We will show below that this leads to a failure mode of the model, in which overconfidence in incorrect output [here,  $\hat{o}(t) = 0$  for  $t \in [0, 2T]$ ] halts learning. Bringing all cases together, we find the loss function

$$\mathcal{L}(r) = \begin{cases} \left| T - \frac{\pi}{2\sqrt{r}} \right| & \text{for } r \geq \frac{\pi^2}{16T^2} = \frac{r^*}{4}, \\ T & \text{otherwise} \end{cases} \quad (20)$$

and the gradient as

$$\nabla \mathcal{L}(r) = \begin{cases} -\frac{\pi}{4r^{3/2}} & \text{for } \frac{\pi^2}{16T^2} < r < \frac{\pi^2}{4T^2}, \\ \frac{\pi}{4r^{3/2}} & \text{for } r > \frac{\pi^2}{4T^2}, \\ 0 & \text{for } r < \frac{\pi^2}{16T^2}. \end{cases} \quad (21)$$

When  $r^* = [\pi^2/(4T^2)]$ , this loss function is zero, achieving the global optimum. Thus, as expected, the loss function does have a minimum at a value  $r^* \sim T^{-2}$ . In this minimum, the state variable  $[\kappa(t)]$  lingers around the origin with negligible changes up to  $T$ , before abruptly shooting off to  $\infty$ . Even though the canonical model has no fixed points for  $r^* > 0$  (and thus for any  $T$ ), the origin resembles one [for  $t \ll T$ ,  $\dot{\kappa} = r^* + \kappa^2 \approx r^*$  leads to negligible changes in  $\kappa(t) \approx 0$ ], hence the name ghost.

### E. Extracting actionable insights about abrupt learning with ghost mechanism

To extract further insights from the analytical loss function and its gradient, we plotted Eq. (20) and numerically calculated loss function values (with finite  $\bar{\kappa}, c$ ) with respect to  $r$  in Fig. 2(a), which revealed three distinct regimes:

- (1) For any  $r < r^*/4$ , loss minimization would enter a no-learning zone and cannot easily recover due to flat loss function values, i.e., zero gradients [ $\nabla \mathcal{L}(r < r^*/4) = 0$ ] in the analytical limit. We call the boundary of this regime,  $r = r^*/4$ , the “point of no return.”
- (2) Though  $r^*$  is a global minimum, the gradient is discontinuous at  $r = r^*$ , i.e.,  $\nabla \mathcal{L}(r \rightarrow r^*)$  does not exist, which leads to oscillatory behavior near the minimum, implying the existence of a critical

learning rate ( $\alpha^*$ ) for a naive gradient descent approach [i.e., updates of the form  $r \rightarrow r - \alpha \nabla \mathcal{L}(r)$ , where  $\alpha$  is the learning rate], beyond which learning stops. For any  $\alpha \geq \alpha^*$ , the training process would eventually enter the no-learning zone (where gradients are exactly zero in the limit  $c \rightarrow \infty$ ) and can no longer recover.

- (3) Picking the learning rate close to  $\alpha^*$  is necessary to achieve learning as the loss function quickly saturates to flat values, i.e.,  $\mathcal{L}(r) \rightarrow T$  for large  $r \gg r^*$ . Since the loss function also changes abruptly for  $(r^*/4) < r < r^*$  (between 0 and  $T$ ), minimizing it would lead to an abrupt decline as  $r \rightarrow r^*$ .

Apart from predicting its existence, our canonical model enables deriving an exact formula for the critical learning rate  $\alpha^*$ . Consider the derivative at  $r = r^*$ ,

$$\text{at } r = \frac{\pi^2}{4T^2} : \nabla \mathcal{L}(r) = \begin{cases} -\frac{2T^3}{\pi^2} & \text{from the left,} \\ \frac{2T^3}{\pi^2} & \text{from the right.} \end{cases} \quad (22)$$

With a learning rate  $\alpha^*$ , the model would be thrown from the global minimum to the point-of-no-return boundary and get stuck in the no-learning zone during the oscillations if it receives the following update:

$$\alpha^* |\nabla \mathcal{L}(r)|_{r=r^*} = \frac{3}{4} r^* = \frac{3\pi^2}{16T^2} \Rightarrow \alpha^* = \frac{3\pi^4}{32} T^{-5}. \quad (23)$$

For any  $\alpha \geq \alpha^*$ , the model would eventually pass the point of no return and output only zeros. Most notably, after rescaling the loss function, and subsequently the gradient, with  $1/T$  to match a mean-squared error, the critical learning rate scales with an inverse power law  $\alpha^* \sim T^{-4}$ ;

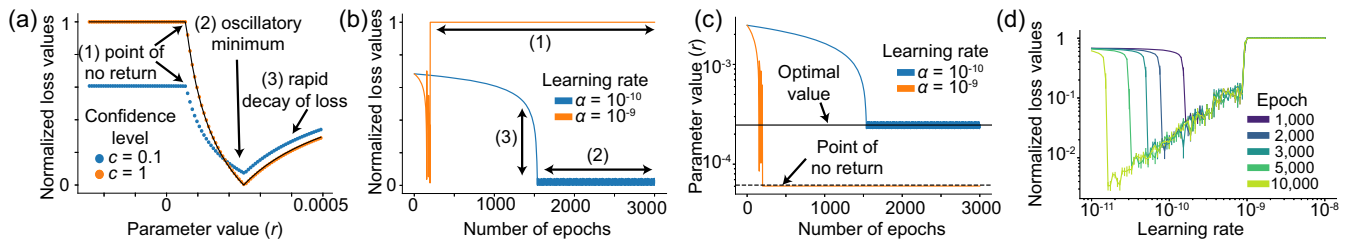


FIG. 2. The canonical model trained on the DA task exhibits abrupt learning and co-occurring learning instabilities. (a) We compared the analytical loss function (black line) vs those computed from realistic parameters (colored dots), in which the model output was defined via a sigmoid function  $\hat{\sigma}(\kappa) = \sigma[c(\kappa - \bar{\kappa})]$ . The loss function had three distinct regimes: (1) a point of no return, (2) a minimum with nonzero gradient, and (3) rapid decay of the loss function for  $r \geq r^*$ , where  $r^* = [\pi^2/(4T^2)]$  is the global minimum. We illustrate in (b) how each regime affects the training. Parameters:  $T = 100$ ,  $\Delta t = 0.1$ , and  $\bar{\kappa} = 10$ . (b),(c) Initializing at  $r := 10r^*$ , we minimized the loss function values using gradient descent with different learning rates, recapitulating all three regimes in learning dynamics. Plots show loss function (b) and parameter values (c) vs number of epochs. Notably, for  $\alpha = 10^{-10}$ , even though the loss function decreases abruptly around epoch 1500, the network does not undergo any bifurcations, as evident from  $r$  not changing its sign during learning. (d) The loss function values normalized by  $T$  at various epochs vs learning rates used for the training process. The canonical model learned best with lower learning rates with more epochs of training. As predicted by Eq. (23), learning is no longer possible for  $\alpha \geq 9 \times 10^{-10}$ . Solid lines, means; error bars, standard error of the mean (SEM) over 10 training instances, in which  $r$  was initialized following a normal distribution that has the mean  $10r^*$  and the standard deviation  $r^*/10$ . (b)–(d)  $T = 100$  and  $\bar{\kappa}, c \rightarrow \infty$  (analytical model).

i.e., longer delay timescales make learning progressively harder and require increasingly fine-tuned learning rates.

To verify the three theoretical insights above, we conducted optimization experiments with naive gradient descent and fixed learning rates [Figs. 2(b)–2(d)], where a single epoch corresponds to a numerical update with the analytically computed gradient. Starting the training from  $r = 10r^*$  and using a learning rate  $\alpha < \alpha^*$ , we observed an abrupt decay of loss function values around epoch 1500, followed by oscillations near the global optimum [Figs. 2(b) and 2(c)]. In contrast, picking a larger learning rate,  $\alpha > \alpha^*$ , pushed the network into the no-learning zone [Figs. 2(b) and 2(c)]. Moreover, varying the learning rate values confirmed that there was indeed a critical learning rate  $\alpha^*$  beyond which learning did not take place [Fig. 2(d)]. The numerical value of the critical learning rate matched the theoretical prediction ( $\alpha^* \approx 9 \times 10^{-10}$ ) from Eq. (23). Even though smaller learning rates led to lower final errors, learning within few epochs was achieved only with learning rates close to criticality,  $\alpha \approx \alpha^*$ .

Finally, since  $r$  values are always positive during learning, the abrupt decays in [Figs. 2(b) and 2(c)] are not preceded by bifurcations, but rather through the emergence of ghost points followed by careful fine-tuning of the scale parameter  $r$ . Interestingly, even though a global minimum with zero loss value exists, the canonical model cannot easily attain it using gradient descent due to complex learning dynamics (Fig. 2), which is not necessarily remedied by using standard optimization strategies, e.g., adding momentum or adaptive methods [48,49]. In fact, both can exacerbate the problem: Momentum accumulates small updates, which can prevent recovery once the system escapes the loss plateau and slightly overshoots  $r \sim r^*$ ; adaptive methods increase the effective step size in flat regions ( $r \gg r^*$ ) by shrinking their gradient history term, which can lead to instability near the parameters that form a ghost ( $r \sim r^*$ ).

Overall, learning instabilities are expected in overparametrized RNNs [9] or deep-learning models solving complex tasks [50]. Here, a simple canonical model with just one state variable and one parameter already captures several complex learning behaviors commonly observed in practice, such as oscillatory minima [see blue lines in Figs. 2(b) and 2(c)] and a range of effective learning rates [Fig. 2(d)].

### III. GHOST DYNAMICS IN LOW-RANK RNNs

To test whether ghosts emerge during training in RNNs and whether their formation follows the predictions of the canonical model, we study the learning dynamics in low-rank RNNs trained to solve working-memory tasks, which we show involve formation of ghosts. Low-rank RNNs provide a natural test bed for this investigation. On the one hand, their dynamics can be reverse engineered and interpreted through low-dimensional phase portraits

[51–53], offering direct access to the mechanisms that emerge during training, i.e., an advantage not available in full-rank counterparts (see Sec. IV). On the other hand, focusing on low-rank dynamics is not overly restrictive: Even full-rank RNNs often converge to low-dimensional solutions [51], and our canonical model already captures local one-dimensional dynamics near ghost points (Sec. II). For these reasons, we begin with rank-one and rank-two RNNs, where interpretability is clearest and direct links to the canonical model can be drawn; whereas Sec. IV will generalize the insights derived from low-rank RNNs to the case of full-rank RNNs.

Specifically, we analyze a representative class of RNNs with the following equations:

$$\tau \dot{\mathbf{x}}(t) = -\mathbf{x}(t) + \tanh[\mathbf{W}\mathbf{x}(t) + \mathbf{W}^{\text{in}}\mathbf{u}(t) + \mathbf{b} + \boldsymbol{\epsilon}], \quad (24)$$

where  $\mathbf{x}(t) \in \mathbb{R}^N$  are firing rates of  $N$  neurons,  $\tau > 0$  is the neuronal decay time, and  $\mathbf{W} \in \mathbb{R}^{N \times N}$  and  $\mathbf{b} \in \mathbb{R}^N$  are trainable weights and biases. Here,  $\mathbf{u}(t) \in \mathbb{R}^{N_{\text{in}}}$  refers to optional control inputs, which we will use for the working-memory tasks below,  $\mathbf{W}^{\text{in}} \in \mathbb{R}^{N \times N_{\text{in}}}$  are corresponding input weights, and  $\boldsymbol{\epsilon}$  is an optional noise term that is set to zero unless explicitly stated. In this section, we focus on rank-one or rank-two RNNs, in which  $\mathbf{W}$  is constrained to have  $N - 1$  or  $N - 2$  zero singular values, respectively.

#### A. Connecting ghost mechanism to rank-one RNNs

Rank-one RNNs are theoretically capable of solving the DA task by virtue of their universal approximation property [53,54]; i.e., they can approximate the canonical model in Eq. (12) with arbitrarily small error. To make this connection explicit, we first enforce a rank-one structure and show that the resulting high-dimensional dynamics reduce to a one-dimensional latent dynamical system. This reduction of Eq. (24) is standard [53] and has also been established in alternative RNN formulations [52,54,55]. Building on this reduction, we then show that the latent system's parameters can be mapped directly onto the scale parameter  $r$  of the canonical model. This mapping links the abstract, analytical ghost dynamics to the concrete training dynamics in low-rank RNNs and provides a principled account of nonlinear learning dynamics, which we will illustrate in the next subsection.

We start by enforcing the rank constraint, i.e., by constructing  $\mathbf{W} = \sum_{k=1}^K \mathbf{m}^{(k)} \mathbf{n}^{(k)T}$  for  $\mathbf{m}^{(k)} \in \mathbb{R}^N$  and  $\mathbf{n}^{(k)} \in \mathbb{R}^N$  and  $k = 1, \dots, K$  denoting the rank. Using this relationship and assuming  $K = 1$  and no inputs for simplicity (we now drop the superscript  $k$ ; derivations for larger ranks  $K > 1$  follow similarly), we can define a latent variable,  $\kappa(t) = \mathbf{n}^T \mathbf{x}(t)$ , such that the RNN equations transform into a one-dimensional form,

$$\tau \dot{\kappa}(t) = -\kappa(t) + \mathbf{n}^T \tanh[\mathbf{m} \kappa(t) + \mathbf{b}] = f(\kappa; \mathbf{m}, \mathbf{n}, \mathbf{b}), \quad (25)$$

which we refer to as the latent dynamical system [53]. In this transformed picture, training the RNN will implicitly update the parameters of the low-dimensional dynamical system, leading to the learning of desired dynamics  $[f(\kappa; \mathbf{m}, \mathbf{n}, \mathbf{b})]$  [53,54]. We do not impose constraints on the latent system itself. Instead, the RNN is optimized to solve the task with a readout defined from the latent variable,  $\hat{o}(t) = \psi[\kappa(t)]$ , where  $\psi(\cdot)$  is a parametrized function (e.g., a sigmoid; see below). This procedure allows us to use the learned parameters  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$  and Eq. (25) to visualize the latent dynamical system that emerges during training. (For further discussions of latent dynamical systems in low-rank RNNs satisfying Eq. (24), see Ref. [53].)

For now, we assume training has converged on a task requiring ghosts, so that a local speed minimum has emerged at  $\kappa = \kappa^*$ . Then, we can write the learned flow near  $\kappa^*$  as

$$\tau \dot{\kappa} = f(\kappa) = \gamma + \beta(\kappa - \kappa^*)^2 + O[(\kappa - \kappa^*)^3], \quad (26)$$

where we recall the connection  $r = \gamma\beta$  from Eq. (11) and denote  $f(\kappa; \mathbf{m}, \mathbf{n}, \mathbf{b})$  concisely as  $f(\kappa)$ . Our goal is to connect  $r$  to the set of parameters  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$ . We first note that the speed minimum at  $\kappa = \kappa^*$  leads to the condition

$$f'(\kappa^*) = 0 \Rightarrow \sum_{i=1}^N n_i m_i [1 - \tanh^2(m_i \kappa^* + b_i)] = 1. \quad (27)$$

The corresponding values of  $\gamma$  and  $\beta$  in terms of  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$  are

$$\gamma = -\kappa^* + \sum_{i=1}^N n_i \tanh(m_i \kappa^* + b_i), \quad (28a)$$

$$\beta = -\sum_{i=1}^N n_i m_i^2 \tanh(m_i \kappa^* + b_i) [1 - \tanh^2(m_i \kappa^* + b_i)]. \quad (28b)$$

Notably,

$$r = \beta\gamma = g(\mathbf{m}, \mathbf{n}, \mathbf{b}, \kappa^*(\mathbf{m}, \mathbf{n}, \mathbf{b})) \quad (29)$$

constitutes a function that depends on the parameters  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$  explicitly, while  $\kappa^*$  depends on these parameters implicitly through the constraint. This function is too complicated to admit a closed-form analytical expression, at least within the knowledge of the authors. Nevertheless, for finite  $N$ , all components are analytic, involving only algebraic operations and the smooth nonlinearity  $\tanh(\cdot)$ . Consequently, the change in  $r := r(\mathbf{m}, \mathbf{n}, \mathbf{b})$  under infinitesimal perturbations  $(\Delta \mathbf{m}, \Delta \mathbf{n}, \Delta \mathbf{b})$  is linear. Formally, by Taylor expansion, we obtain

$$\begin{aligned} r(m_1 + \Delta m_1, \dots, n_1 + \Delta n_1, \dots, b_1 + \Delta b_1, \dots) \\ = r(m_1, \dots, n_1, \dots, b_1, \dots) + \Delta r + O(\|(\Delta \mathbf{m}, \Delta \mathbf{n}, \Delta \mathbf{b})\|_2^2), \end{aligned} \quad (30)$$

with the first-order increment

$$\Delta r = \sum_{i=1}^N \frac{\partial r}{\partial m_i} \Delta m_i + \sum_{i=1}^N \frac{\partial r}{\partial n_i} \Delta n_i + \sum_{i=1}^N \frac{\partial r}{\partial b_i} \Delta b_i. \quad (31)$$

Each partial derivative contains both explicit contributions from  $\mathbf{m}$ ,  $\mathbf{n}$ ,  $\mathbf{b}$ , and implicit contributions through  $\kappa^*(\mathbf{m}, \mathbf{n}, \mathbf{b})$ . Thus, for sufficiently small parameter changes  $(\Delta \mathbf{m}, \Delta \mathbf{n}, \Delta \mathbf{b})$ , e.g., under a small learning rate, Eq. (31) describes how parameter updates induce a change in  $r$ . Thus, a gradient descent on  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$  would change the parameters of the low-rank RNN, but can we connect the resulting change to the gradient-driven parameter updates of the ghost mechanism?

Fortunately, Eq. (31) provides the necessary link between  $\Delta r$  and  $\nabla \mathcal{L}(r)$ . To see why, note that the one-dimensional reduction near the ghost is assumed to capture the observed neural activities during task performance. Since the loss is computed from these activities, we assume that perturbations of  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$  that leave  $r$  invariant leave the loss approximately unchanged, so that  $\mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b}) \approx \mathcal{L}(r)$ . Now, we recall that the canonical model prescribes this change directly as

$$\Delta r = -\alpha \nabla \mathcal{L}(r), \quad (32)$$

with learning rate  $\alpha$ . In the low-rank RNN, by contrast,  $r$  is not updated directly but through parameter updates,

$$\Delta m_i = -\alpha \frac{\partial \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})}{\partial m_i}, \quad (33a)$$

$$\Delta n_i = -\alpha \frac{\partial \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})}{\partial n_i}, \quad (33b)$$

$$\Delta b_i = -\alpha \frac{\partial \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})}{\partial b_i}, \quad (33c)$$

for  $i = 1, \dots, N$ . Substituting Eq. (33) into Eq. (31) connects the gradient dynamics in Eq. (32) to the learning dynamics of low-rank RNN by

$$\nabla \mathcal{L}(r) \approx \sum_{\theta \in (m_i, n_i, b_i)} h_{\theta}(\mathbf{m}, \mathbf{n}, \mathbf{b}) \frac{\partial \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})}{\partial \theta}, \quad (34)$$

for coefficients  $\mathbf{h}(\mathbf{m}, \mathbf{n}, \mathbf{b}) = \nabla_{\theta} r$  [where  $\nabla_{\theta}$  refers to the gradient with respect to the parameters  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$ ] that depend on the parameters forming the ghost in the first place. This equation is simply the inner product of the

coefficient vector  $\mathbf{h}(\mathbf{m}, \mathbf{n}, \mathbf{b})$  and the gradient with respect to RNN parameters  $\nabla_{\theta} \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})$ . Thus, the update to the canonical parameter  $r$  can be written as a projection of the high-dimensional gradient of low-rank RNN parameters,

$$\begin{aligned} \Delta r &= -\alpha \nabla \mathcal{L}(r), \\ &\approx -\alpha \mathbf{h}(\mathbf{m}, \mathbf{n}, \mathbf{b})^T \nabla_{\theta} \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b}), \\ &= \mathbf{h}(\mathbf{m}, \mathbf{n}, \mathbf{b})^T \Delta \theta, \end{aligned} \quad (35)$$

where  $\Delta \theta = -\alpha \nabla_{\theta} \mathcal{L}(\mathbf{m}, \mathbf{n}, \mathbf{b})$  refers to the parameter update in the rank-one RNN. This equation establishes a direct linear connection between the update of the canonical parameter  $\Delta r$  and the update of the low-rank RNN parameters  $\Delta \theta$ : The former is obtained as a projection of the latter onto  $\nabla_{\theta} r$ .

In essence, Eq. (35) is an application of the chain rule to the dependency  $r := r(\mathbf{m}, \mathbf{n}, \mathbf{b})$ , relying on the analyticity of this mapping and holding under the conditions of small learning rate (which is necessary for learning a large timescale; as derived in Sec. II) and correspondingly small parameter updates. Under these assumptions, vanishing slopes, ill-conditioned directions, or bifurcations that hinder learning in the canonical dynamics are likewise expected to appear in the learning dynamics of the low-rank RNN parameters as we show next.

### B. Reproducing theoretical results from the canonical model

We now turn to the learning dynamics of rank-one RNNs trained to solve the DA task and compare them to the predictions of the canonical model. For this task, we define the RNN output as

$$\hat{o}(t) = \sigma[c(\kappa - \bar{\kappa})], \quad (36)$$

for some  $\bar{\kappa} > 0$ . Training the network corresponds to minimizing the difference between  $o(t)$  and  $\hat{o}(t)$  while updating the parameters  $(\mathbf{m}, \mathbf{n}, \mathbf{b})$ . Unless otherwise stated, for simplicity, we fix  $\bar{\kappa} = 1$  (without loss of generality) and  $c = 10$ , though we will also report training results with learnable  $c$  below.

As a first step, we confirmed that rank-one RNNs exhibit the ghost mechanism to solve the DA task when trained with a suitable learning rate [Fig. 3(a)]. During learning, we observed the emergence of a (local) ghost point [Fig. 3(a)], which coincided with sharp transitions in both the loss function (abrupt decay) and the  $L_2$  norm of the gradient (abrupt increase). Importantly, the latent variable near the ghost point had very slow time dynamics, leading to slow changes in latent activations. By fine-tuning the distance at the ghost point [similar to  $r$  in Eq. (12)], RNN learns to wait for a delay period of  $T$  and output one only after the latent variable escapes the ghost point. When we studied the development of this ghost point in the latent dynamical system, we uncovered that the latent dynamical system has always had one fixed point (there was only one fixed point after every epoch within  $\kappa \in [-15, 15]$ ). Consequently, as in our canonical model, this RNN created the ghost point without undergoing any bifurcations, but by carefully fine-tuning the minimum speed value at the ghost.

Similar to the canonical model, the learning dynamics in rank-one RNNs also showed the oscillatory behavior for low loss function values [Fig. 3(a)]. Yet, unlike the canonical model, we observed that the rank-one RNN was able to eventually escape the oscillations and decrease the loss further. Studying the learned latent dynamical system

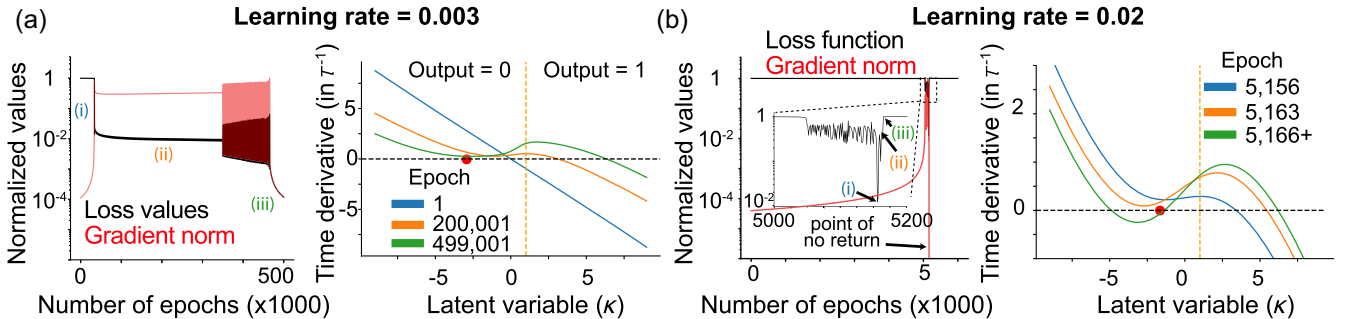


FIG. 3. A rank-one RNN trained on the DA task reproduced the main findings of the canonical model. We trained a rank-one RNN on the DA task, in which the output of the RNN was defined as  $\hat{o}(\kappa) = \sigma[c(\kappa - 1)]$ . Here,  $\kappa$  is the latent variable and  $c$  is the confidence level. (a) Left: the RNN trained with a relatively low learning rate showed the abrupt jump in the loss function [between (i) and (ii)] and had oscillatory behavior before converging to a minimum [between (ii) and (iii)]. Right: the resulting network learned local ghost points with a small, but nonzero, distance from the  $y = 0$  line. (b) When training the same network with higher learning rate, a saddle-node bifurcation occurred, putting the network beyond the point of no return. The network could no longer recover, as indicated by the practically zero gradient after the bifurcation. Parameters:  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $T = 100$  ms,  $N = 100$  neurons,  $c = 10$ . We sampled initial RNN parameters following  $m_i \sim \mathcal{N}(0, N^{-1})$ ,  $n_i \sim \mathcal{N}(0, N^{-1})$ , and  $b_i = 0$  for  $i = 1, \dots, N$  neurons. Here,  $\mathcal{N}(\mu, \sigma^2)$  refers to a normal distribution with mean  $\mu$  and variance  $\sigma^2$ . We initialized all units to be  $x_i(0) = -0.3$  and used stochastic gradient descent. Red dots correspond to the initial values of  $\kappa(t)$  for the final networks. “Gradient norm” refers to the  $L_2$  norm of the (flattened) gradient computed over all trainable parameters. Training videos for multiple seeds are available via the Zenodo link in Ref. [56].

revealed that the escape was accompanied with a sharpening of the curvature around the ghost [Fig. 3(a)], whereas our canonical model had a fixed curvature regardless of  $r$ . With the increased curvature,  $\kappa(t)$  spent less time in uncertain states around  $\bar{\kappa}$ , i.e., those with  $\hat{\delta} \approx 0.5$ , and the output abruptly transitioned from zero to one. Though prior work has shown that networks with many parameters are more expressive compared to their counterparts [54], this observation suggests an added benefit: They may have desirable loss landscapes to achieve the global minima (which may exist but remain unattainable) in less expressive models, echoing earlier work on Gaussian random fields in high-dimensional spaces [57].

Next, we increased the learning rate to test the prediction of criticality. As predicted, this prevented learning altogether [Fig. 3(b)]. Specifically, we examined the latent dynamical-system evolution in a rank-one RNN trained with a high learning rate [Fig. 3(b)]. Similar to the canonical model, the network underwent a saddle-node bifurcation and remained stuck in a no-learning zone with a nearly zero gradient. Though one might think that the fixed nature of the output confidence was the reason, we observed that another network, in which  $c$  was trainable, still got stuck in the no-learning zone (see the data and code available via the Zenodo link in Ref. [56]).

In summary, low-rank RNNs, whose high-dimensional neural activities are described by a one-dimensional latent dynamical system, displayed all three properties predicted by the canonical model: Abrupt loss decreases with ghost formation, oscillatory minima near the optimum, and a critical learning rate that can drive the network into a no-learning zone.

### C. Ghost mechanism in working-memory tasks

Having established the ghost mechanism in the simple delayed-activation task, we next asked whether similar dynamics also emerge in more demanding working-memory paradigms [58,59]. We focused on two canonical tasks: delayed cue discrimination (DCD) and delayed match to sample (DMTS). In both tasks, the RNN receives an input cue of value  $\pm 1$  for a duration  $T_{\text{inp}}$ , followed by a delay interval  $T_{\text{delay}}$  during which the network must retain this information. In the DCD task, after the delay, the RNN is required to output  $\pm 1$  corresponding to the original cue for a response window  $T_{\text{resp}}$ . In the DMTS task, a second cue is presented after the delay for another  $T_{\text{inp}}$  interval, and the RNN must then report whether the two cues match by producing 1 (match) or 0 (nonmatch) during the response window  $T_{\text{resp}}$ .

Although rank-one RNNs might, in principle, solve these tasks, in practice, they failed to converge to accurate solutions. We therefore considered rank-two RNNs, whose activity (assuming no noise) is effectively captured by a two-dimensional latent dynamical system, which was at

times driven by some control inputs  $\mathbf{u}(t) \in \mathbb{R}^2$  depending on the task,

$$\begin{aligned} \tau \dot{\mathbf{\kappa}}_j(t) &= -\mathbf{\kappa}_j(t) + \mathbf{n}^{(j)T} \tanh(\mathbf{z}(t)), \\ \mathbf{z}(t) &= \sum_{i=1}^2 \mathbf{m}^{(i)} \mathbf{\kappa}_i(t) + \mathbf{W}^{\text{in}} \mathbf{u}(t) + \mathbf{b}, \end{aligned} \quad (37)$$

where the latent variables are defined as  $\mathbf{\kappa}_j(t) = \mathbf{n}^{(j)T} \mathbf{x}(t)$  for  $j = 1, 2$ . The output is read out from latent variables via  $\hat{\delta}(t) = \psi[\mathbf{w}_{\text{out}}^T \mathbf{\kappa}(t) + b_{\text{out}}]$ , where  $\mathbf{w}_{\text{out}} \in \mathbb{R}^2$  and  $b_{\text{out}} \in \mathbb{R}$  are output weight and bias parameters,  $\psi$  is task-dependent output nonlinearity (tanh for DCD,  $\sigma$  for DMTS, Fig. 4). Training the RNN parameters therefore corresponds to updating the parameters of this low-dimensional system, just as in the one-dimensional setting of Eq. (25). We used this formulation to visualize and study the learned latent dynamics via two-dimensional phase portraits in the absence of control inputs (Fig. 4).

In both tasks, training trajectories exhibited abrupt loss drops proceeded by oscillatory gradient norms, coinciding with the emergence and subsequent tuning of the ghost mechanism (Fig. 4). For the DCD task, two sharp loss transitions appeared within the first 8000 epochs [Fig. 4(a), markers (i)–(iii)]. Examining the latent phase portraits at these transition points revealed distinct mechanisms underlying each jump [Figs. 4(b)–4(d)]. During the first jump, between (i) and (ii), the RNN underwent a bifurcation and acquired bistable dynamics with attractors corresponding to the go and no-go outputs [Fig. 4(b), middle]. These new fixed points enabled discrimination of the input cues, but because no mechanism yet existed to delay the output, the RNN produced premature responses [Fig. 4(c), first panel]. The second jump in the loss, between (ii) and (iii), led to the formation of two cue-specific ghosts [Fig. 4(b), right]. These bottlenecks prolonged trajectories in a cue-dependent manner, suppressing premature outputs during the delay and enabling the timed transition during the response window [Fig. 4(c), second panel]. In later epochs, marked as (iv) and (v) in Fig. 4(a), the loss decreased gradually as the timescales near the ghosts were fine-tuned [Fig. 4(d), left] and, eventually, extended slow regions formed [Fig. 4(d), right]. Ultimately, these changes allowed the RNN to delay its output for precisely the required duration [Fig. 4(c), last two panels].

A similar sequence of learning processes unfolded in the learning of the DMTS task [Figs. 4(e)–4(h)]. In the early epochs, marked as (i)–(iii) in Fig. 4(e), training again exhibited abrupt loss changes followed later by oscillatory gradient norms. At this stage, the latent dynamics failed to reliably separate match from mismatch trajectories (Figs. 4(f) and 4(g)). While several transient ghosts emerged in this window [Fig. 4(f)], these were not yet arranged to maintain or compare cue information across or after the delay, respectively. As a result, network outputs

resembled the behavior of the simpler delayed-activation task, responding in the same way regardless of the input patterns rather than performing the intended cue comparison [Fig. 4(g), first two panels].

In later epochs, the network had acquired the ability to correctly distinguish and compare the cues [Fig. 4(g), last two panels]. At this stage, several oscillatory cycles had formed in the latent phase portraits, which enabled cue comparison, with ghosts positioned in their vicinity to slow

down the dynamics as needed [Fig. 4(h)]. This configuration gated progression within the latent dynamics, forcing trajectories to linger near a ghost until the second cue arrived, i.e., either knocking them out to a larger cycle that produced a positive output or leaving them confined to smaller cycles that yielded zero outputs [Fig. 4(h), right]. In this way, the network preserved information about the initial cue during the delay and generated a properly timed match vs no-match response after the second stimulus [Fig. 4(g), final panel].

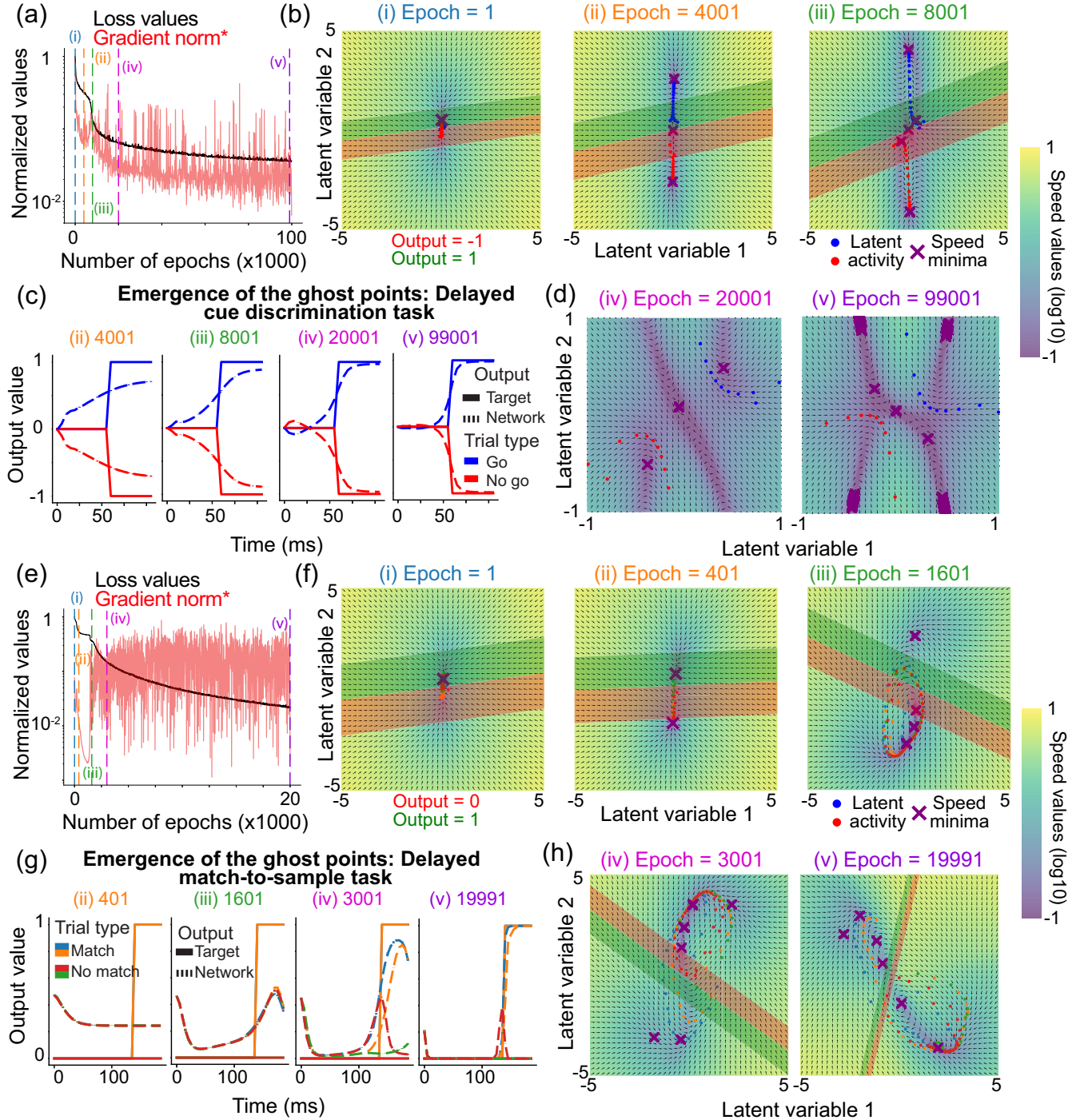


Fig. 4. (Caption on next page.)

FIG. 4. Rank-two RNNs trained on two commonly studied working-memory tasks exhibited ghost mechanisms. (a)–(d) A rank-two RNN was trained on a delayed cue discrimination task, where the network observed one of two cues (+1 for go, −1 for no-go) and, after a short delay, produced an output indicating the cue identity. (a) Training dynamics were characterized by abrupt transitions, oscillatory gradients, and sharp spikes in the loss values. Five representative epochs were marked with colored labels and discussed in subsequent panels. Data points for every 100 epochs are shown. “Gradient norm\*” denotes the sum of Frobenius norms of gradients for the low-rank components of the recurrent weight matrix. (b) Latent phase portraits revealed distinct mechanisms underlying these transitions. Between (i) and (ii), the network learned a bistable dynamical system, with attractors corresponding to go and no-go output states. Between (ii) and (iii), the RNN learned two trial-type-specific ghost points, prolonging latent trajectories near the decision boundary. Flow speed is color coded, arrows show normalized flow directions, red and green bands denote the decision band [ $\hat{o} \in [-0.5, 0.5]$  for a  $\tanh(\cdot)$  output nonlinearity and  $\hat{o} \in [0.27, 0.73]$  for a  $\sigma(\cdot)$  output nonlinearity], purple crosses mark speed minima (identified numerically from the phase portraits with nearest-neighbor comparisons), and blue and red circles track latent states for representative go and no-go trials. (c) Plots show network outputs across distinct epochs. Dashed lines denote averages within trials of the same type; the SEM values are negligible. (d) A close-up of the latent phase portrait at two later epochs, marked with (iv) and (v), in which ghost timescales are fine-tuned and extended slow regions emerge, respectively. (e)–(h) Same as in (a)–(d), but for a rank-two RNN trained on a delayed match-to-sample task, in which the network receives two input cues ( $\pm 1$ ) separated by a delay and has to output whether these cues match (1) or not (0). Similar to before, RNN underwent several regimes of learning [denoted as (i–iv)] in (e), with each mechanism (f) endowing the network output with a unique capability (g). In the final solution, ghosts emerged around and on an oscillatory cycle to delay the network output and to aid matching of the two cues (h). Parameters:  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $N = 100$  neurons. Initial neural activity values (batch size of 500 for each trial type) were sampled as  $x_i(0) \sim \tanh(\mathcal{N}(0, 0.3^2))$ . Recurrent weights for the rank-two networks are defined as  $\mathbf{W} = \mathbf{M}\mathbf{N}$ , with initializations  $M_{ij} \sim \mathcal{N}(0, 1)$ ,  $N_{ji} \sim \mathcal{N}(0, N^{-1})$  for  $i = 1, \dots, N$  and  $j = 1, 2$ , such that  $\mathbf{W}$  is constrained to be rank two. Input and output weights are initialized using Xavier uniform initialization. Neuronal biases are initialized via  $b_i \sim \mathcal{N}(0, 0.1^2)$  for  $i = 1, \dots, N$  and the output bias is initialized by sampling uniformly from  $[-1/\sqrt{2}, 1/\sqrt{2}]$ . For (a)–(d), the network was trained with gradient descent (learning rate  $10^{-3}$ );  $T_{\text{inp}} = 10$  ms,  $T_{\text{delay}} = 50$  ms, and  $T_{\text{resp}} = 50$  ms. No additional noise was added to the neurons during forward evolution. For (e)–(h), the network was trained with ADAM [48] (learning rate  $10^{-4}$ );  $T_{\text{inp}} = 30$  ms for both input windows,  $T_{\text{delay}} = 80$  ms, and  $T_{\text{resp}} = 50$  ms. A random noise with zero mean and  $10^{-2}$  s.d. was added to individual neural activities at every time step. Training videos for multiple seeds are available via the Zenodo link in Ref. [56].

Taken together, our results on the DCD and DMTS tasks suggest that rank-two RNNs can acquire working-memory behavior through transient, ghost-induced bottlenecks in a two-dimensional latent space. These bottlenecks emerged alongside loss jumps and enabled short-lived memory via slow passage near ghosts, providing an effective delay without trapping the dynamics in stable attractors. Thus, although our study of the delayed-activation task may appear to be a highly constrained scenario, ghosts arise naturally across distinct working-memory tasks.

#### IV. INSIGHTS FOR FULL-RANK RNNs

We now extend our analysis to full-rank RNNs, where lifting the rank restriction expands both the parameter space and the dimensionality of the underlying dynamical system, enabling the networks to explore higher-dimensional manifolds and exhibit qualitatively richer behaviors [30]. In this section, we first illustrate that ghosts can emerge in full-rank RNNs during learning. Then, we continue by studying three key aspects that reveal both shared principles and distinctive features between low- and full-rank RNNs. First, both exhibit critical learning rates beyond which learning fails, though full-rank networks reach criticality at higher values and display more stable dynamics overall. Second, oscillatory gradients and associated no-learning zones emerge in both cases, demonstrating the generality of these phenomena. Third, we demonstrate that these no-learning zones arise from the properties of the

learned dynamical system and its outputs, rather than architectural assumptions. By designing an output mapping that controls confidence levels, we show that networks can escape no-learning zones and resume learning, validating our mechanistic understanding.

##### A. Ghost dynamics in full-rank RNNs

We now turn to full-rank RNNs by lifting the rank restriction on the weight matrix  $\mathbf{W} \in \mathbb{R}^{N \times N}$  and study full-rank networks whose dynamics are described by Eq. (24). Prior studies have shown that such networks often develop speed minima that are not fixed points, known as “slow point manifolds,” when trained on working-memory tasks [10,45,60–62]. We now show that full-rank RNNs develop the same ghost bottlenecks introduced in Sec. II A and demonstrated in low-rank networks (Figs. 3 and 4). These bottlenecks constitute a special form of slow points that funnel nearby trajectories through transient slow regions in state space and can be implemented without explicit dimensionality restrictions.

To start with, we equip the full-rank RNNs with readouts

$$\hat{o}(t) = \psi(\mathbf{w}_{\text{out}}^T \mathbf{x}(t) + b_{\text{out}}), \quad (38)$$

where  $\psi(\cdot)$  is a fixed nonlinearity: sigmoid (DA and DMTS tasks) or tanh (DCD tasks); and readout is now taken from the full population via  $N$ -dimensional weights  $\mathbf{w}^{\text{out}} \in \mathbb{R}^N$  with  $b^{\text{out}} \in \mathbb{R}$  being the output bias. For an

autonomous RNN, whether fixed, slow, and ghost points exist depends on the subset of parameters  $\theta = \{W, b\}$ . To identify such speed minima, we analyzed autonomous dynamics by setting all inputs to zero, as task inputs are sparse. Following [45], we define an RNN-specific energy function as,

$$s_{\theta}(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \left[ -x_i + \tanh \left( \sum_{\alpha=1}^N W_{i\alpha} x_{\alpha} + b_i \right) \right]^2, \quad (39)$$

which we minimize over  $\mathbf{x}$  while holding  $\theta$  fixed. Since this nonconvex optimization can yield multiple minima, we initialize  $\mathbf{x}$  with neural activities observed during normal network operation and perform minimization using a PYTORCH-based model. A detailed implementation is

provided in the shared codebase (see the data and code availability statement at the end). Using this procedure, we reverse-engineered the inner workings of two representative RNNs as they performed DCD and DMTS tasks, paralleling our discussion in Fig. 4 for low-rank RNNs.

For the full-rank RNN trained on the DCD task (Figs. 5(a) and 5(b)), training dynamics showed sharp loss decreases within the first approximately 200 epochs, accompanied by oscillatory gradients. Extracting speed minima from neural trajectories revealed the emergence of two new fixed points as the underlying state-space phenomenon during this transition. This bistable dynamical system, similar to the low-rank RNN in Figs. 4(a)–4(d), enabled the network to map input cues to the correct outputs. At later stages, refinements in learning introduced a ghost point, identified through the eigenvalue spectrum of

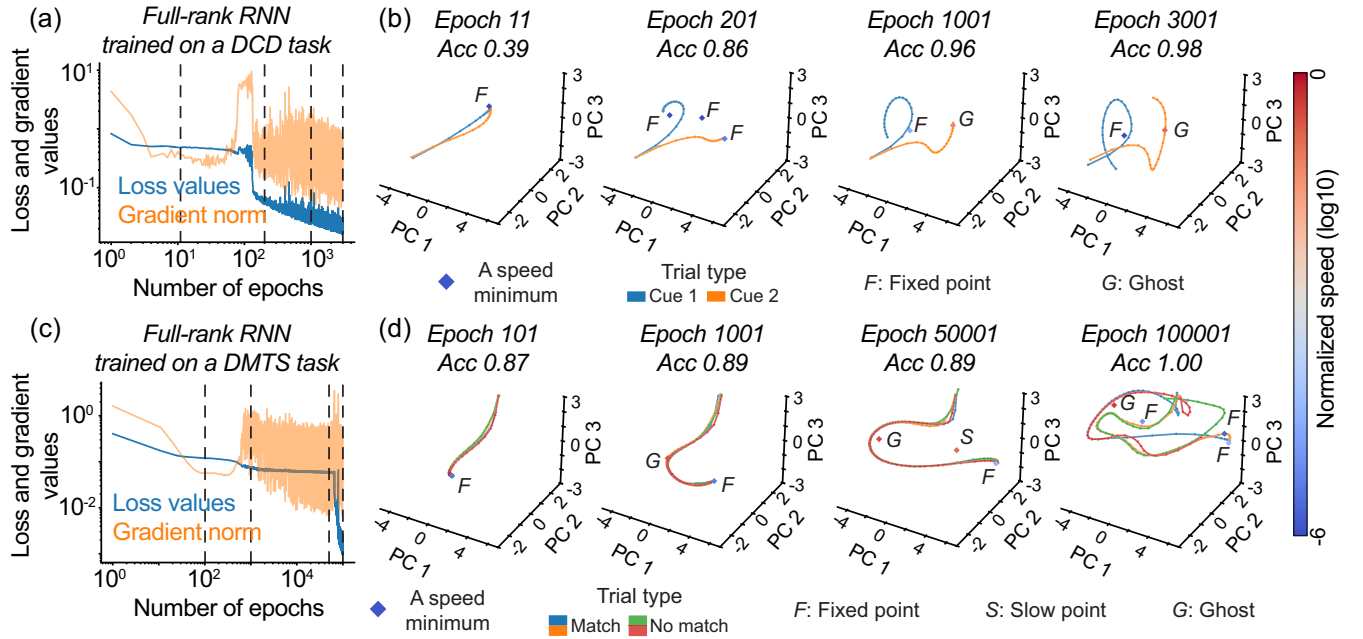


FIG. 5. Representative full-rank RNNs exhibited ghost mechanisms when trained on working-memory tasks. To illustrate that full-rank RNNs can learn ghost mechanism to support working memory, we studied a full-rank RNN trained on a DCD task (a),(b) and another one trained on a DMTS task (c),(d). (a) The learning progression curves as a function of epochs. Gradient norm refers to the Frobenius norm of the gradient corresponding to the recurrent weight matrix. Most significant jumps occur within the first approximately 100–200 epochs, with subsequent epochs refining the learning solution and gradually decreasing the loss function values. (b) We extracted the speed minima from the neural activities of full-rank RNNs and studied their progression over learning epochs. The plots show the neural trajectories projected down to the top three principal components (PC 1–PC 3). Diamonds belong to speed minima found by initializing our numerical solver with the samples along the trajectories. The two speed minima at epochs 1001 and 3001 (with orange colors) are ghosts, characterized by each Jacobian having one effectively zero and otherwise all negative eigenvalues. Others are fixed points (though all are not necessarily attractive). The color scale on the right denotes the normalized speed ( $\|\dot{\mathbf{x}}\|_2$ ) of the fixed, slow, or ghost points. (c),(d) Same as in panels (a),(b), but for an RNN performing a DMTS task. Notably, this RNN also utilized a slow point during learning transitions, which is distinct from ghosts in the sense that its Jacobian can have positive eigenvalues (compare panels (b) and (d)). Parameters:  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $T_{\text{inp}} = 10$  ms,  $T_{\text{delay}} = 50$  ms/150 ms for panels (b) and (d), respectively,  $T_{\text{resp}} = 50$  ms, and  $N = 100$  neurons. Firing rates are initially sampled following  $x_i(0) \sim \tanh(\mathcal{N}(0, 0.1^2))$ , iid noise with an s.d. of  $10^{-2}$  is added to each neuron during every step of the time evolution. We initialized the recurrent, input, and output weights using Xavier uniform initialization [63]. Neuronal biases are initialized from the uniform distribution on  $[-1, 1]$ , whereas the output bias is taken from the uniform distribution on  $[-0.1, 0.1]$ . We trained the networks with gradient descent, each batch containing a single trial from each type and  $\alpha = 10^{-2}$ . Accuracies (Acc.) denote the average agreement between ternarized (thresholds  $\pm 0.33$ ; (a),(b)) or binarized (threshold 0.5; (c),(d)) network outputs and target outputs across all times and trials.

the Jacobian computed from Eq. (39). Specifically, this Jacobian exhibited one (approximately) zero eigenvalue and otherwise negative values, in line with our definition of a ghost in Sec. II A. This ghost prolonged trajectories through a transient slow region for one cue, while for the other trial type, the RNN delayed its response by curving trajectories in the high-dimensional state space [blue vs orange lines in the final two panels of Fig. 5(b)]. Thus, both the ghost-induced bottleneck and the curved neural trajectory enabled the network to suppress premature responses and generate appropriately timed outputs, with the former closely paralleling the progression of learned mechanisms observed in low-rank RNNs [Figs. 4(a)–4(d)].

For the full-rank RNN trained on the DMTS task [Figs. 5(c) and 5(d)], training dynamics again showed abrupt loss transitions and oscillatory gradients. By epoch 1001, the network had developed a ghost in its state space that indiscriminately delayed responses for all trials [Fig. 5(d); first two panels]. Trajectories around this ghost closely resembled the curved shapes observed in low-rank RNNs [Figs. 4(e)–4(h)]. As training progressed, a slow point also emerged: A local minimum of the energy function that, unlike a ghost, can have positive eigenvalues in its Jacobian spectrum [Fig. 5(d), third panel]. Yet, with continued training, the slow point was not found and the network introduced additional fixed points. These fixed points, together with the existing ghost, were fine-tuned to gate trajectories during the delay period before the arrival of the second cue, eventually yielding a correctly timed match vs no-match response [Fig. 5(d), final panel]. Thus, the full-rank RNN trained on the DMTS task relied on a combination of fixed points, slow

points, and ghosts to learn working-memory computations, paralleling the mechanisms observed in low-rank RNNs [Figs. 4(e)–4(h)].

Overall, while earlier work established that full-rank RNNs rely on speed minima to perform working-memory tasks [45], our results show that ghosts, as defined in Sec. II A, also emerge in full-rank networks and play roles analogous to those in low-rank counterparts. There is, however, a key difference in interpretability: Low-rank RNNs permit far more direct, analytical insights into the underlying mechanisms (Figs. 3 and 4), whereas in full-rank networks, the same structures are present but harder to disentangle and require numerical analysis (Fig. 5).

### B. Partial stabilization of critical learning rates with loosely constrained ranks

In our canonical model of ghost mechanism, which was trained to perform the DA task (Sec. II), we identified a critical learning rate above which learning failed; lower rates led to eventual convergence, but rates just below this threshold yielded the fastest early progress [Fig. 2(d)]. Consistent with this, when we studied rank-one RNNs with outputs defined as in Eq. (36) that utilized the ghost mechanism to solve this task (see Fig. 3), we found the same qualitative pattern [Fig. 6(a), left]. Full-rank RNNs, in contrast, have access to more expressive resources (e.g., additional loosely constrained ranks), which could, in principle, stabilize training dynamics. As before, full-rank RNNs continued to exhibit a critical learning rate [Fig. 6(a), right], but the added ranks did substantially stabilize and improve the learning dynamics: The critical learning rate increased by nearly 2 orders of magnitude, and rates near

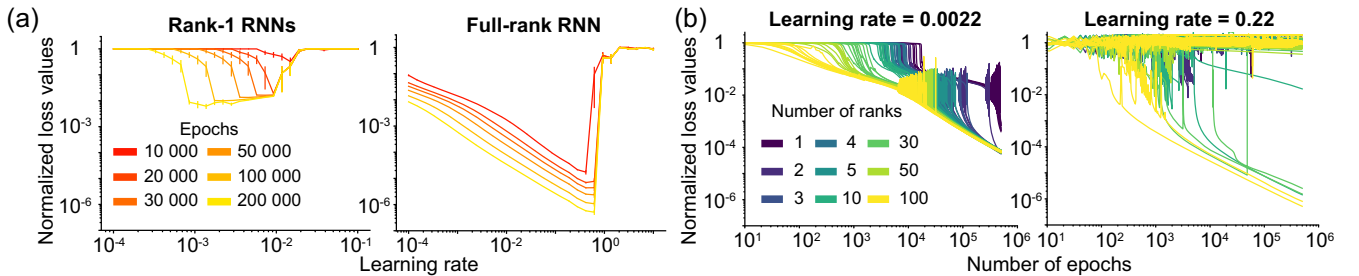


FIG. 6. Increasing the ranks of the RNNs available for training partially stabilized learning dynamics in the DA task. (a) We performed the analysis in Fig. 2(d) for the rank-one (left) and full-rank (right) RNNs trained on the DA task. We reported means and standard errors over ten random seeds. (b) We trained RNNs with varying ranks (1, 2, 3, 4, 5, 10, 30, 50, 100) on the DA task. Each line corresponds to a run with a different seed. Left. The RNN trained with a relatively low learning rate showed the abrupt jump in the loss function values and the following oscillations, similar to Fig. 3(a). Increasing the number of trainable ranks shortened the learning process, with milder and sustained decreases in the loss function. Right. For a substantially high learning rate, few RNNs that could train without entering the no-learning zone had higher ranks (here,  $K \geq 10$ ). This result contrasts with the  $K = 1$  case that is sufficient to solve the task. See the Zenodo repository in Ref. [56] for additional data involving varying learning rates, where lower-rank networks progressively fail to train with increasing  $\alpha$ . Parameters:  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $T = 100$  ms, and  $N = 100$  neurons. We initialized all units to be  $x_i(0) = -0.3$  for  $i = 1, \dots, N$  and used stochastic gradient descent. For rank-one RNNs, we set  $c = 10$ . For (a), rank-one RNNs were identical to those in Fig. 3 and had the same initialization procedure. The full-rank RNNs were initialized with  $W_{ij} \sim \mathcal{N}(0, N^{-1})$ ,  $b_i = 0$ ,  $(w_{\text{out}})_i \sim \mathcal{N}(0, N^{-1})$ , and  $b_{\text{out}} \sim \mathcal{N}(0, N^{-1})$ . For (b), the readouts are taken from the full population  $\mathbf{x}(t)$  across all ranks with  $\psi(\cdot) = \sigma(\cdot)$  output nonlinearity. The networks were initialized similar to full-rank RNNs, except for  $M_{ij} \sim \mathcal{N}(0, N^{-1})$  and  $N_{ij} \sim \mathcal{N}(0, N^{-1})$ .

this value became optimal for sustained training (reminiscent of the “catapult mechanism” studied in a feedforward context in Ref. [27]).

We next examined learning curves across different rank configurations [Fig. 6(b)] and found that increasing the number of trainable ranks in  $\mathbf{W}$  consistently resulted in learning with fewer epochs. Notably, full-rank RNNs exhibited the loss jump up to 2 orders of magnitude faster compared to their rank-one counterparts [Fig. 6(b); initial loss jumps happen rapidly in all cases, note the log scale of the  $x$  axis]. While some of the observed improvements in training speed may arise from changes in the effective initialization induced by varying rank constraints (which is an interesting direction for future work on initialization strategies for low-rank RNNs), we found that higher-rank RNNs were substantially less likely to become trapped in no-learning zones, particularly at larger learning rates [Fig. 6(b)]. Despite this stabilization, full-rank RNNs still entered and often remained in no-learning zones and occasionally encountered oscillatory minima [Fig. 6(b)]. Thus, while loosely constrained ranks helped mitigate learning instabilities, they did not fully eliminate them. In summary, full-rank RNNs performing the DA tasks can still have oscillatory minima and exhibit a critical learning rate beyond which learning ceases, though adding ranks increases the range of effective learning rates and improves the training stability.

### C. Oscillatory gradients and no-learning zones in full-rank RNNs performing working-memory tasks

Encouraged by the results on the DA tasks, we next considered full-rank RNNs trained to perform DMTS and DCD tasks. Notably, RNNs often blend in ghosts within other strategies to solve these tasks (Figs. 4 and 5); therefore, it is, so far, an unanswered question whether these other mechanisms make RNNs immune to the no-learning zones. Here, we test this explicitly by training RNNs to perform these tasks across a range of learning rates.

We started with the DMTS task and, as before, observed the emergence of a critical learning rate in this task [Fig. 7(a)], beyond which learning failed. Interestingly, networks trained on this task showed abrupt improvements in their loss functions, often accompanied by large (and sometimes oscillatory) changes in the gradient norm [Fig. 7(b)]. Moreover, in line with Fig. 6, these full-rank RNNs displayed minimal oscillations near the minima, and in some cases, these oscillations were absent altogether [Fig. 7(b), left]. At high learning rates, gradients remained small even when averaged over the full training window, indicative of no-learning zones [Fig. 7(a)]. When examining individual training instances, we indeed found no-learning zones, in which gradients were effectively zero [Fig. 7(b)]. Next, we performed the same experiment with full-rank RNNs performing DCD tasks, which

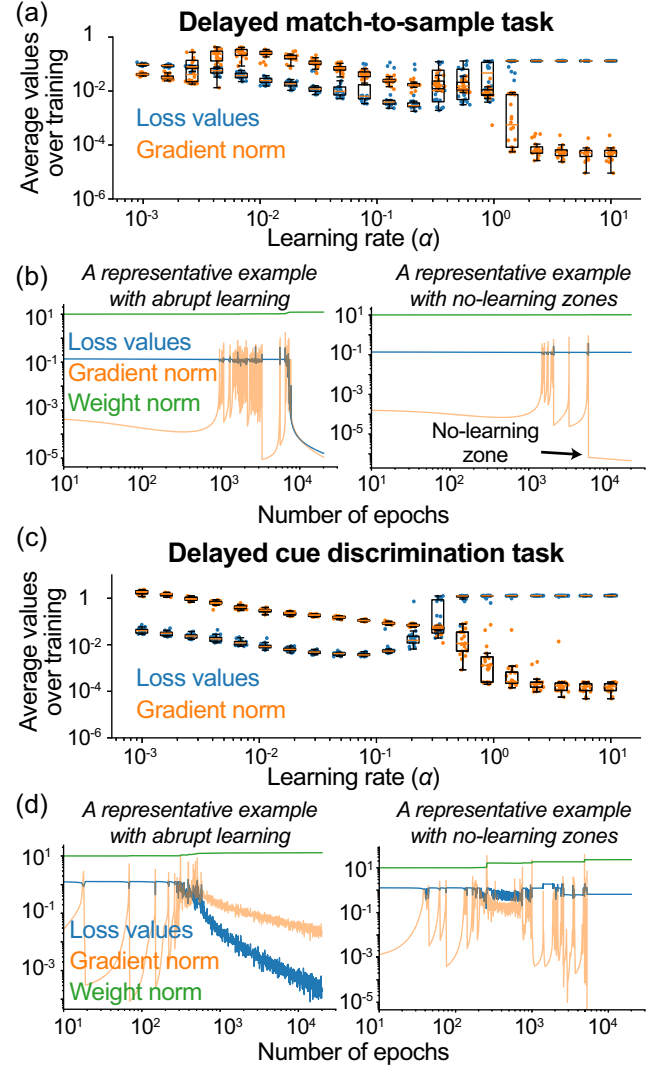


FIG. 7. Full-rank RNNs exhibit both oscillatory gradients and no-learning zones. (a),(b) Full-rank RNNs trained on DMTS tasks. (a) Loss function values and Frobenius norms of recurrent weight gradients as a function of learning rate. Each dot represents a training run with a different seed ( $N_{\text{epochs}} = 20\,000$ ), averaged over all epochs. (b) Learning dynamics for two example runs,  $\alpha \approx 1.44$ . The green solid line corresponds to the Frobenius norm of the recurrent weights during training. (c),(d) Same as in (a),(b), but for full-rank RNNs performing DCD tasks. For (d), we used two representative runs trained with the learning rate  $\alpha \approx 0.34$ . Parameters:  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $T_{\text{inp}} = 30$  ms,  $T_{\text{delay}} = 80$  ms, and  $T_{\text{resp}} = 50$  ms, and  $N = 100$  neurons. All firing rates were initially zero and no noise was added during time evolution. RNN parameters were initialized following the same procedure as in Fig. 5 and training ensued with gradient descent (each batch contains one trial per type).

qualitatively agreed with our findings from the DMTS task [Figs. 7(c) and 7(d)].

In standard deep learning, vanishing gradients typically originate from saturating neuronwise activation functions and are addressed using piecewise-linear activations [64].

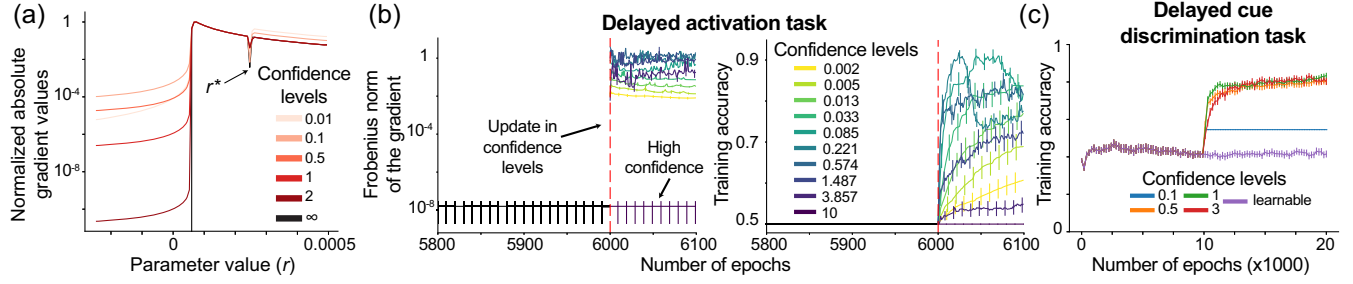


FIG. 8. Lowering the spurious confidence allows recovering from the no-learning zone. (a) To illustrate how lowering output confidence enables gradients to build beyond the point of no return in the ghost mechanism, we plotted the gradients of the canonical model with respect to its scale parameter  $r$ , otherwise using the same parameters as in Fig. 2(a). Notably, excessively low confidence leads to incorrect outputs, indicating the existence of an optimal confidence level for training (here, around  $c \approx 0.1$ ). For infinite confidence, the gradient is exactly zero for  $r < r^*/4$ . (b) To test the confidence lowering in practice, we trained 100 rank-one RNNs with a confidence  $c = 10$  and the learning rate  $\alpha = 0.02$  for 6000 epochs, otherwise using the same parameters as in Fig. 3. 74 out of 100 RNNs learned the task at some point, whereas 48 of them entered the no-learning zone (quantified as having training accuracy of  $\leq 0.5$  for the last 50 epochs). We further trained these RNNs after lowering the confidence levels, which allowed them to recover. Left: the Frobenius norm of the flattened gradient over all parameters. Right: the training accuracy as a function of number of epochs. Solid lines, means; error bars, SEM over 48 networks that were in the no-learning zones within the epochs [5950, 6000]. Training accuracy is defined as the agreement between discretized network outputs and target outputs (cf. Fig. 5). (c) To generalize the confidence lowering to higher-rank RNNs, we trained full-rank RNNs on DCD tasks with  $\alpha = 0.1$  and compared the training dynamics of fixed vs learnable output confidence. During the first 10000 epochs,  $c$  is trainable in both cases. Solid lines, means; shaded areas, SEM over 100 networks. Parameters for (c):  $\tau = 10$  ms,  $\Delta t = 5$  ms,  $T_{\text{inp}} = 10$  ms,  $T_{\text{delay}} = 50$ , and  $T_{\text{resp}} = 50$  ms, and  $N = 100$  neurons. Firing rates were initialized with  $x_i \sim \tanh[\mathcal{N}(0, 0.1^2)]$ , random noise (with s.d.  $10^{-3}$ ) added to the RNN at every time step, and training with gradient descent with 20 trials (10 per cue) using the ADAM optimizer [48]. RNNs were initialized following the same procedure as in Fig. 5.

However, analysis of the RNN exhibiting a no-learning zone in Fig. 7(b) (data available via Zenodo [56], refer to code designated to Fig. 7) revealed that most firing rates lay outside the saturation regime of  $\tanh(\cdot)$ ; that is, the hidden unit activations were not saturated. Thus, the vanishing gradients observed here stem from a different source: As dynamical systems, RNNs are sensitive to small parameter changes that can qualitatively alter their internal dynamics. Confident outputs inherited from earlier epochs can trap the network in a state where the output remains confidently incorrect, blocking further learning. (See, for example, Fig. 3(b), where postbifurcation dynamics remain stuck in negative  $\kappa$  values, yielding a confident zero output.)

Overall, our findings here reveal that, in full-rank RNNs performing complex behavioral tasks, critical learning rates and no-learning zones emerge as generic consequences of how dynamical systems navigate optimization landscapes. Notably, no-learning zones arise from dynamical failure modes rather than the architectural or numerical limitations typically encountered in deep learning. In the next section, we make this observation concrete.

#### D. Lowering spurious confidence encourages recovery from no-learning zones

Given the inevitability of the no-learning zones, which appear whenever learning rates exceed critical values, we next set out to identify a mechanism through which they can arise. In earlier literature, vanishing gradients were extensively studied and attributed to the repeated

multiplication of Jacobian matrices in saturated regimes of nonlinearities, which drives gradients toward zero [20]. A natural question is whether the no-learning zones observed here reflect the same phenomenon or, instead, emerge from a distinct mechanism.

To answer this question, we return to our canonical model solving the DA task (Fig. 2) and analyze why no-learning zones arise. Recall that, with infinite confidence ( $c = \infty$ ), the model outputs 0 or 1 depending on whether the state variable  $\kappa(t)$  exceeds a threshold  $\bar{\kappa} \gg 1$ . For a given  $T$ , the optimal solution scales as  $r^* \propto T^{-2}$  [Eq. (20)], yielding outputs of 0 for the first  $T$  time steps and 1 thereafter. As discussed in Sec. II, this scaling complicates not only training for large  $T$ , but also places  $r^*$  close to the value  $\tilde{r} := r^*/4$ , where the output remains identically zero throughout the full  $2T$  duration. For  $r < \tilde{r}$ , variations in  $r$  no longer affect the output, which stays zero despite infinitesimal parameter changes. The loss function therefore becomes flat with zero local gradient.

If oscillations around the optimum or stochastic fluctuations drive the system into this regime, learning halts, independent of whether neural activities are saturated. The resulting no-learning zones are thus distinct from the classical vanishing gradient problem: They arise not from architectural properties such as Jacobian contraction in saturated nonlinearities, but from the structure of the underlying dynamical system, where small parameter shifts [e.g., from  $\tilde{r} + \delta$  to  $\tilde{r} - \delta$  for some small  $\delta$ ; black line in Fig. 8(a)] can induce qualitatively different outputs. This abrupt change reflects the infinite confidence of the

canonical model. By contrast, fixing confidence to finite values during training preserves small but nonzero gradients even within the no-learning zone, allowing eventual recovery [Fig. 8(a)]. In other words, the no-learning zones originate from the overconfidence of dynamical models in their outputs: Small parameter changes in the dynamics can globally alter predictions, leaving training trapped in incorrect yet overly confident solutions.

We next tested this prediction in practice with rank-one RNNs trained on the DA task, for which we have already established no-learning zones for large enough learning rates (Fig. 3). Specifically, we asked whether artificially lowering confidence levels can help them escape no-learning zones. To test this, we returned to rank-one RNNs performing DA tasks and lowered their output confidence  $[c, \text{Eq. (36)}]$  after they were stuck in no-learning zones [Fig. 8(b)]. As expected, we found that reducing confidence allowed the RNNs to become unstuck and relearn the task [Fig. 8(b)], which aligns with the predictions of the canonical model, in which lower confidence values permit small but nonzero gradients to form beyond the point of no return [Fig. 8(a)].

The results from the canonical model and rank-one RNNs suggest a general principle, which we next tested in full-rank RNNs performing DCD tasks. To set and fix the output confidence level in full-rank RNNs, we defined the readout as follows:

$$\hat{o}(t) = \tanh\left(c \frac{\mathbf{w}_{\text{out}}^T \mathbf{x}(t) + b_{\text{out}}}{\|\mathbf{w}_{\text{out}}\|_1}\right), \quad (40)$$

where the division by  $\|\mathbf{w}_{\text{out}}\|_1$  ( $\mathcal{L}_1$  norm) introduces scale invariance to  $\mathbf{w}_{\text{out}}$ . This modification effectively fixes the confidence levels of the  $\tanh(\cdot)$  nonlinearity, similar to Eq. (36) for the rank-one RNNs.

To better approximate realistic training conditions, we trained full-rank RNNs using batches and the ADAM optimizer [48], introduced noise into the firing rates, and randomized the initial state across trials. With this setup, we trained full-rank RNNs using a high learning rate that could produce no-learning zones [Fig. 8(c), purple line]. As expected, RNNs trained under this condition failed to improve output accuracy [Fig. 8(c)]. Then, we manually intervened at epoch 10 000 by lowering and fixing the output layer's confidence [Fig. 8(c)]. Following this intervention, output accuracy increased compared to the baseline condition [Fig. 8(c)], in some cases even reaching near perfect performance. However, excessively low confidence values again prevented learning from being sustained in the long run.

## V. DISCUSSION

In this work, we introduced a canonical model that provides an analytical description of abrupt learning through the formation of ghost points. This model is not

merely illustrative, but rather an exact reduction of high-dimensional dynamics near ghosts, recapitulating their learning dynamics in an architecture-agnostic manner. With this model, we derive an analytical scaling law for the critical learning rate beyond which formation of ghosts becomes impossible. While there are likely other mechanisms for learning long-term dependencies [20], this theoretical discovery provides one important piece of the puzzle for why longer working-memory tasks seem to require increasingly fine-tuned optimization; i.e., the solution, and not necessarily the architecture, requires it. We validated these theoretical predictions across both low- and full-rank RNN architectures, which involved both the predicted oscillatory gradients and no-learning zones. Furthermore, we identified two practical strategies for stabilizing ghost-based learning: increasing the number of trainable ranks, which partially stabilizes learning at higher learning rates, and reducing output confidence, which mitigates the associated vanishing gradient problem in full-rank networks.

### A. Why does the ghost mechanism lead to abrupt learning?

An important aspect of the ghost mechanism is its locality in both dynamics and the scale parameter. Losing sight of this fact, and studying Fig. 2(a), may lead to an incorrect interpretation, in which ghosts emerge only within a small range of the scale parameter  $r$ . Moreover, it is not necessarily clear why  $r$  should start from large positive values rather than, say, negative values. The key insight is that we are interested in networks that are randomly initialized, as in Fig. 3. As shown for rank-one RNNs, random initialization is unlikely to have any meaningful fixed-point structure in place [65]. Instead, as learning progresses, the ghost emerges spontaneously at a specific location [Fig. 3(a)], and this emergence is rapidly governed as  $r$  approaches  $O(r^*)$  from a regime with no fixed points at all, i.e., from large  $r$  values. In this regime, the gradient rapidly changes from flat to very high values as  $r$  approaches  $O(r^*)$  following Eq. (21), causing learning to occur abruptly.

That being said, the loss function in Eq. (20) describes only the local behavior of the scale parameter when the speed minimum is already formed. Based on the success of training across various conditions and initial seeds (Fig. 3), there are likely many parameter configurations in high-dimensional space that can map current RNN parameters to a local ghost mechanism, i.e., following Eq. (29). On the other hand, the large gradients emerging after entering a parameter region governed by the ghost mechanism are simultaneously both the blessing and curse of optimization. For learning rates that scale as  $T^{-4}$  for delay duration  $T$  (and therefore vanish when long-term dependencies must be learned), these large gradients can trigger saddle-node bifurcations and eventual entrapment in no-learning zones.

This dual nature of the loss function clarifies why the ghost mechanism can lead to both abrupt learning of new capabilities and entrapment in regions of parameter space with vanishing gradients.

### B. A dynamics-first approach to studying learning dynamics

Much of the machine-learning literature approaches learning dynamics by first selecting specific architectures that exhibit phenomena of interest (e.g., abrupt learning), then analyzing their behavior [9,10,28]. However, this architecture-first strategy can limit our ability to extract universal principles of computation and learning. Here, we adopt a “dynamics-first” perspective by focusing on problems solved by dynamical systems, specifically working-memory tasks. We find that one common solution, the ghost mechanism, enforces universal constraints on both the time evolution of state variables and the learning dynamics of network parameters. These constraints enabled us to predict diverse phenomena across architectures: the emergence of oscillatory gradients, the existence of critical learning rates that scale with delay duration, and the formation of no-learning zones where optimization fails entirely. By deriving these predictions from the dynamical solution rather than architectural specifics, we obtain results that transfer from our simple canonical model to low- and full-rank RNNs.

This success of the dynamics-first approach is particularly relevant for systems neuroscience, where the goal is often not to replicate the brain’s exact architecture but to understand computational principles that govern neural processing [52,53,59,67–71]. When RNNs are trained to match neural recordings or perform behavioral tasks, they serve as “digital twins” that capture functional dynamics, which can be tested causally with experimental manipulations [72], rather than replicating biological details at the level of individual neurons [73]. These models are valuable precisely because they abstract away from implementation details to reveal computational motifs. In this sense, neuroscientists have implicitly embraced a dynamics-first approach, without necessarily articulating the associated mathematics. Our canonical reduction makes this philosophy explicit and precise: By isolating the ghost mechanism, we identify one fundamental dynamical structure that endows finite-dimensional continuous-time dynamical systems with the ability to maintain information over time, regardless of whether those systems are artificial or biological.

Indeed, ghost formation itself is directly relevant for computational neuroscience: Dynamical bottlenecks, like ghosts shown in Figs. 4 and 5, are routinely observed in trained RNNs, i.e., *in silico* models of neural computation [53,60,68], and may underlie short-term memory in neural circuits [47]. Previous work has typically identified these structures *post hoc*, using numerical methods to analyze

fully trained networks [45]. Our approach complements this by revealing how such bottlenecks form during training, exposing both their computational utility and the learning instabilities they create. The canonical model in Eq. (11) thus provides a conceptual bridge between abstract dynamical-systems theory and empirical observations in neuroscience. Specifically, for dynamical systems to learn ghosts, learning rates should decrease as the timescale of learned computations increases. Working-memory tasks with variable delay periods provide a natural setting to test this prediction. We anticipate two key behavioral signatures: (i) Animals should exhibit lower effective learning rates when trained on longer delay periods, and (ii) interventions that artificially increase learning rates (e.g., through pharmacological manipulation) may paradoxically impair learning by driving network parameters into no-learning zones, resulting in accuracy plateaus long after the intervention ends. Testing these predictions on behaving animals requires observing and modifying correlates of learning rates in brains, toward which promising research efforts have long been devoted in the field [74–76].

Finally, this dynamics-first framework also suggests that when neuroscientists observe slow dynamics or persistent activity in neural recordings [77–80], these phenomena may not simply reflect arbitrary network properties but could arise from fundamental constraints on how general dynamical systems learn to perform temporal computations. The ghost mechanism predicts specific signatures in both dynamics (transient slowdowns near critical points) and learning (oscillatory gradients, critical learning rates), which could be tested in biological learning experiments or in the optimization of RNN models of neural data [51,81]. Such tests would help determine whether ghost points represent a universal solution that evolution and learning converge upon when faced with the challenge of maintaining information over time, and if so, what other solutions are out there.

### C. Ghost signatures persist despite increasing architectural complexity

Although the ghost mechanism we describe here is fundamentally one-dimensional [30], RNNs are known to utilize higher-dimensional slow point manifolds (i.e., geometric analogs of ghost points) to solve memory tasks [34,45,61]. This is an area of growing interest in computational neuroscience [82], in which high-dimensional state spaces of full-rank RNNs enable a broader repertoire of mechanisms (beyond ghostlike attractors) to construct and sustain such manifolds, raising the possibility that the ghost dynamics observed in our model may reflect artifacts of low-rank constraints or task simplicity, and not necessarily one central mechanism of abrupt learning associated with delaying network outputs. This is certainly true to some extent, as we have observed that higher dimensions contain new strategies for delaying network outputs

(see Figs. 4 and 5 with the curved neural activities; also refer to Ref. [83]). Yet, as we show in Fig. 5, full-rank RNNs still do learn ghosts and, as we show in Fig. 7, the core insights from our simplified model generalize to the learning dynamics of full-rank RNNs performing complex working-memory tasks.

Our progression from simple to complex systems reveals a fundamental trade-off between interpretability and generality. In the canonical model, we achieved complete mechanistic understanding: exact scaling laws, precise conditions for instability, and analytical expressions for critical parameters. Moving to low-rank RNNs (Fig. 4), we could still trace how specific dynamical objects (ghosts, fixed points, slow points) contribute to computation, visualizing the complete phase portrait and identifying each component's functional role. However, in full-rank networks (Fig. 5), our analysis necessarily becomes more limited. While we can identify ghost formations and observe characteristic neural trajectories in principal component projections, the full mechanistic picture remains opaque. This progression mirrors a broader challenge in the field: We often need to trade analytical tractability for biological realism.

This trade-off has shaped how the field of computational neuroscience studies high-dimensional RNNs. Earlier work has already made attempts at characterizing these systems, though even then the search has been for low-dimensional features (as in Fig. 5) that can be studied with known numerical methods [45]. As evidenced by existing literature, this is not always overly restrictive, since in many cases full-rank RNNs do learn low-dimensional, effectively low-rank, solutions [51,84]. That being said, characterizing the full range of solutions learned by full-rank RNNs, or predicting which degenerate solution will be learned to solve a particular task in general networks [67,85], remains a major mystery studied across the board. Our work contributes to this effort; i.e., at least one mechanism, the ghost, has appeared consistently across this complexity gradient as long as the demands of the tasks lead to ghost formation.

#### D. Outlook: Broader implications

Our analysis of ghost mechanisms reveals fundamental constraints on how dynamical systems learn temporal computations, with implications spanning neuroscience, machine learning, and biological learning. For machine learning, our findings illuminate why training RNNs on temporal tasks remains challenging despite architectural advances. Previous work has shown that RNNs often converge to effectively low-rank solutions [51,84,86], achieved through two distinct strategies: (1) restricting updates to low-rank components from the outset, as in reservoir computing [25] and traditional low-rank training [51,52,54] or (2) allowing unconstrained updates that gradually refine into a low-rank structure during learning

(see Ref. [87] for recent related ideas in the deep-learning literature). Our analysis suggests strategy (2) may inherently stabilize learning by providing additional degrees of freedom during training, even when the final solution is low dimensional. This insight could inform the design of training algorithms that delay rank constraints until later in training, while explicitly leveraging high-dimensional parameter spaces early in the training to navigate around no-learning zones before converging to simpler representations.

Looking forward, the ghost mechanism exemplifies how careful analysis of minimal models can yield insights that scale to complex systems. As the field develops increasingly sophisticated architectures for temporal processing, from transformers [41] to state-space models [88], understanding fundamental dynamical constraints becomes ever more critical. Beyond recurrent systems, feedforward architectures like deep neural networks and large language models exhibit analogous abrupt transitions marking the acquisition of new computational capabilities [14–19]. Whether ghost mechanisms or fundamentally different dynamical structures produce these transitions in non-recurrent systems remains an important open problem. Extending mechanistic analyses of this kind to spiking neural networks represents a further important direction for bridging theoretical understanding with biologically plausible implementations [89–91].

The ghost framework demonstrates that some learning challenges arise not from implementation details, but from how any dynamical system must reorganize to maintain information over time. Future work should explore whether similar canonical reductions can illuminate other persistent challenges in learning dynamics, ultimately bridging the gap between theoretical understanding and practical advances in both artificial and biological intelligence. In particular, investigating whether critical learning rates, no-learning zones, and oscillatory gradient dynamics emerge in these modern architectures could reveal whether the constraints we identified are truly universal or specific to RNN-like dynamics.

One particularly compelling case concerns architectures that introduce delayed inputs directly (most notably, attention-based ones [41,92]), which stands in contrast to recurrent systems, which tend to maintain temporal context in their internal dynamics (e.g., via ghosts). This observation leads to a concrete, falsifiable prediction: Modern models may not be subject to the same inverse power-law scaling of critical learning rates with delay duration identified here. More broadly, future work should systematically study the effects of diverse heuristics in the machine-learning literature (e.g., alternative loss functions, regularization strategies, and choices of nonlinearity and model architecture), which may reshape the optimization landscape in ways that interact nontrivially with ghost formation and the associated instabilities.

Finally, our findings have implications for future research in physics, biophysics, and neuroscience. First, dimensions in parameter space that have little effect on the system's output or loss are universally observed in many physical models [93]. Our findings suggest that one reason for their existence may be the need to stabilize the learning dynamics of few well-constrained dimensions. Second, in the traditional deep-learning paradigm, the confidence levels could be optimized, not artificially controlled, and therefore the training process would be susceptible to no-learning zones. We presented a potential remedy in Fig. 8, i.e., different mechanisms may be crucial for learning (practicing) vs inference (performing). This observation underscores the importance of incorporating mechanisms beyond loss minimization (e.g., simulated annealing [94]) to modulate system behavior when learning stalls in neural networks and mirrors seminal work on adult zebra finches, where males exhibit variability during practice, but perform with precision and reduced neural variability in the presence of a female [95,96]. The parallel suggests that biological systems may have evolved similar strategies to avoid dynamical traps during learning, providing a concrete prediction about how neural circuits might modulate confidence during skill acquisition.

### ACKNOWLEDGMENTS

We would like to thank Dr. Marta Blanco-Pozo, Dr. Saeed Ahmed Khan, Dr. Sarah Kushner, and Dr. Daniel Kunin for their insightful comments on the manuscript; Dr. Nina Miolane, Abby Bertics, and members of the Geometric Intelligence Laboratory at UC Santa Barbara for their helpful feedback on an earlier version of the project; and Dr. Boris Shraiman for fruitful discussions on the loosely constrained parameters and learning in high-dimensional dynamical systems. M. J. S. gratefully acknowledges funding from the Simons Collaboration on the Global Brain, the Vannevar Bush Faculty Fellowship Program of the U.S. Department of Defense, and Howard Hughes Medical Institute. F.D. receives funding from Stanford University's Mind, Brain, Computation and Technology program, which is supported by the Stanford Wu Tsai Neuroscience Institute. Internships of E. C. and B. K. were supported in part by a grant from the Feldman-McClelland Open-a-Door fund of the Pittsburgh Foundation. B. K. thanks the Impact Scholarship and Research Scholarship for Critical Thinkers programs from the Bridge to Türkiye Funds for supporting his visit to Stanford University. Some of the computing for this project was performed on the Sherlock cluster. We would like to thank Stanford University and the Stanford Research Computing Center for providing computational resources and support that contributed to these research results. This research was supported in part by Grant No. NSF PHY-2309135 and the Gordon and Betty Moore Foundation

Grant No. 2919.02 to the Kavli Institute for Theoretical Physics (KITP).

### DATA AVAILABILITY

The data that support the findings of this article are openly available [56].

### APPENDIX: METHODOLOGICAL DETAILS FOR REPRODUCIBILITY

#### 1. Task details

In this paper, we investigate three key tasks to understand the nature of the ghost mechanism and evaluate its generalizability across a spectrum of RNNs, from one rank to full rank, performing behavioral tasks. To ensure clarity and reproducibility, we describe all three tasks in detail in this appendix.

The “delayed-activation” task is the first of the three tasks examined using both the toy model and RNNs. This task consists of two distinct intervals: the delay interval ( $T_{\text{delay}}$ ) and the response interval ( $T_{\text{resp}}$ ). Unlike the other tasks studied in this work, the DA task does not involve any input signal. Instead, the desired output is defined by the following piecewise function  $o(t)$ :

$$o(t) = \begin{cases} 0 & \text{if } t \in \{T_{\text{delay}}\}, \\ 1 & \text{if } t \in \{T_{\text{resp}}\}. \end{cases} \quad (\text{A1})$$

The “delayed match-to-sample” task involves both input  $u(t)$  and output  $o(t)$  signals. This task consists of four distinct periods: the input interval ( $T_{\text{inp}}$ ), delay interval ( $T_{\text{delay}}$ ), match interval (also  $T_{\text{inp}}$ ), and response interval ( $T_{\text{resp}}$ ). The task features two cue periods: the input and match intervals. After receiving the input cue (randomly chosen from  $\{-1, 1\}$ ) during the input interval, the model must retain this information through the delay interval. During the match interval, a second cue is presented. The model must determine whether this cue matches the original input. In the response interval, the model should output 1 if the cues match, and 0 otherwise. Formally, the input and output signals are defined as follows:

$$u(t) = \begin{cases} \pm 1 & \text{if } t \in \{T_{\text{inp}}, T_{\text{match}}\}, \\ 0 & \text{if } t \in \{T_{\text{delay}}, T_{\text{resp}}\}, \end{cases} \quad (\text{A2})$$

$$o(t) = \begin{cases} 0 & \text{if } t \in \{T_{\text{inp}}, T_{\text{delay}}, T_{\text{inp}}\}, \\ 1 & \text{if } t \in \{T_{\text{resp}}\} \text{ and } u_{\text{inp}} = u_{\text{match}}, \\ 0 & \text{if } t \in \{T_{\text{resp}}\} \text{ and } u_{\text{inp}} \neq u_{\text{match}}. \end{cases} \quad (\text{A3})$$

The “delayed cue discrimination” task also involves both an input  $u(t)$  and an output  $o(t)$  signal. However, unlike the DMTS task, it does not include a match interval; only the input interval serves as the cue presentation.

The objective of the task is to retain the input cue through the delay period and produce the corresponding response afterward. Formally, the input and output signals are defined as follows:

$$u(t) = \begin{cases} \pm 1 & \text{if } t \in \{T_{\text{inp}}\}, \\ 0 & \text{if } t \in \{T_{\text{delay}}, T_{\text{resp}}\}, \end{cases} \quad (\text{A4})$$

$$o(t) = \begin{cases} 0 & \text{if } t \in \{T_{\text{inp}}, T_{\text{delay}}\}, \\ u_{\text{inp}} & \text{if } t \in \{T_{\text{resp}}\}. \end{cases} \quad (\text{A5})$$

Here  $u_{\text{inp}}$  refers to the input presented during the input window at the particular trial.

## 2. Analyses with RNNs

There are several important distinctions between the trained RNNs and our toy model. First, for large  $\kappa$ , the time evolution equations asymptotically becomes

$$\tau \dot{\kappa}(t) \rightarrow -\kappa(t) + O(1), \quad |\kappa| \rightarrow \infty, \quad (\text{A6})$$

where  $O(1)$  refers to the  $\kappa$ -independent saturation term due to the saturation of  $\tanh(\cdot)$  nonlinearities. Thus, unlike the toy model, RNN equations cannot generate a global ghost point, as there is always at least one fixed point to accommodate  $f(\kappa; \mathbf{m}, \mathbf{n}, \mathbf{b}) \rightarrow -\kappa$  limit as  $\kappa \rightarrow \pm\infty$ . Instead, as we show in Fig. 3, the ghost point emerges locally.

Second, the RNN equations have a characteristic time-scale  $\tau$ , with respect to which we perform an Euler discretization and design the delay period  $T$ . Specifically, we pick  $\Delta t = 5$  ms such that  $\Delta = \Delta t / \tau = 0.5$  and the time evolution equation becomes

$$\mathbf{x}[t + \Delta t] = (1 - \Delta)\mathbf{x}[t] + \Delta \tanh(\mathbf{z}[t]), \quad (\text{A7})$$

where  $\mathbf{z}[t] = \mathbf{W}\mathbf{x}[t] + \mathbf{W}^{\text{in}}\mathbf{u}[t] + \mathbf{b} + \boldsymbol{\epsilon}$ .

A rank-one RNN, despite being characterized by a single dynamical variable, has  $O(N)$  parameters, which means there are far more knobs to turn, effectively creating redundancy and loosely constrained parameters. Thus, we do not expect the critical learning rate in RNNs to follow Eq. (23), as the RNNs are more expressive than the toy model. For example, they can learn to change the curvature of the ghost point as shown in Fig. 3(a). However, as shown in Fig. 6(a), we did recover the same trend between the learning rates and the final loss function values.

For our analysis with increasing ranks in Fig. 6, we used the RNNs described by Eq. (24), but with the following output:

$$\hat{o}(t) = \sigma[\mathbf{w}_{\text{out}}^T \mathbf{x}(t) + b_{\text{out}}], \quad (\text{A8})$$

where  $\mathbf{w}_{\text{out}} \in \mathbb{R}^N$  and  $b_{\text{out}} \in \mathbb{R}$  are trainable parameters. Unlike the rank-one RNN settings in earlier figures, this choice allows one to train the confidence parameter ( $c$  for the previous models) and the threshold value. We enforce the rank constraints by defining  $\mathbf{M} \in \mathbb{R}^{N \times K}$  and  $\mathbf{N} \in \mathbb{R}^{K \times N}$  such that

$$\mathbf{W} = \mathbf{M}\mathbf{N} \quad (\text{A9})$$

is at most a rank- $K$  matrix. We train these networks by first computing the forward evolution using the same discretization procedure explained in Eq. (A7) and performing stochastic gradient descent on the discretized version of the loss function in Eq. (14).

For our analyses on working-memory tasks, presented in Figs. 5, 7, and 8, we used two distinct readouts for full-rank RNNs. For the DMTS task, we used the same network output as in Eq. (A8). For the DCD task, since the output is contained within  $[-1, 1]$ , we used a  $\tanh(\cdot)$  nonlinearity in the readout layer in Eq. (A8). For Fig. 4, we instead computed the readouts from the two-dimensional latent variables as opposed to the neural activities, with the task-specific output nonlinearities as described above.

- 
- [1] Donald Olding Hebb, *The Organization of Behavior: A Neuropsychological Theory* (Psychology Press, New York, 2005).
  - [2] Eric R. Kandel, James H. Schwartz, Thomas M. Jessell, Steven Siegelbaum, A. James Hudspeth, Sarah Mack *et al.*, *Principles of Neural Science* (McGraw-Hill New York, 2000), Vol. 4.
  - [3] Anne E. Urai, Brent Doiron, Andrew M. Leifer, and Anne K. Churchland, *Large-scale neural recordings call for new insights to link brain and behavior*, *Nat. Neurosci.* **25**, 11 (2022).
  - [4] Logan G. Wright, Tatsuhiro Onodera, Martin M. Stein, Tianyu Wang, Darren T. Schachter, Zoey Hu, and Peter L. McMahon, *Deep physical neural networks trained with backpropagation*, *Nature (London)* **601**, 549 (2022).
  - [5] Peter L. McMahon, *The physics of optical computing*, *Nat. Rev. Phys.* **5**, 717 (2023).
  - [6] Sunil Pai, Zhanghao Sun, Tyler W. Hughes, Taewon Park, Ben Bartlett, Ian AD Williamson, Momchil Minkov, Maziyar Milanizadeh, Nathnael Abebe, Francesco Morichetti *et al.*, *Experimentally realized in situ back-propagation for deep learning in photonic neural networks*, *Science* **380**, 398 (2023).
  - [7] Demis Hassabis, Dharshan Kumaran, Christopher Summerfield, and Matthew Botvinick, *Neuroscience-inspired artificial intelligence*, *Neuron* **95**, 245 (2017).
  - [8] Anthony M. Zador, *A critique of pure learning and what artificial neural networks can learn from animal brains*, *Nat. Commun.* **10**, 3770 (2019).
  - [9] Lukas Eisenmann, Zahra Monfared, Niclas Göring, and Daniel Durstewitz, *Bifurcations and loss jumps in RNN*

- training, *Adv. Neural Inf. Process. Syst.* **36**, 70511 (2023).
- [10] Udith Haputhanthri, Liam Storan, Yiqi Jiang, Adam Shai, Hakki Orhun Akengin, Mark Schnitzer, Fatih Dinc, and Hidenori Tanaka, *Why do recurrent neural networks suddenly learn? Bifurcation mechanisms in neuro-inspired short-term-memory tasks*, in *ICML 2024 Workshop on Mechanistic Interpretability* (ICML 2024 Workshop on Mechanistic Interpretability, Vienna, Austria, 2024).
- [11] Peter DelMastro, Rushiv Arora, Edward Rietman, and Hava T. Siegelmann, *On the dynamics of learning time-aware behavior with recurrent neural networks*, [arXiv:2306.07125](https://arxiv.org/abs/2306.07125).
- [12] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso *et al.*, *Beyond the imitation game: Quantifying and extrapolating the capabilities of language models*, [arXiv:2206.04615](https://arxiv.org/abs/2206.04615).
- [13] Sanjeev Arora and Anirudh Goyal, *A theory for emergence of complex skills in language models*, [arXiv:2307.15936](https://arxiv.org/abs/2307.15936).
- [14] Dingli Yu, Simran Kaur, Arushi Gupta, Jonah Brown-Cohen, Anirudh Goyal, and Sanjeev Arora, *Skill-mix: A flexible and expandable family of evaluations for AI models*, [arXiv:2310.17567](https://arxiv.org/abs/2310.17567).
- [15] Ekdeep Singh Lubana, Kyogo Kawaguchi, Robert P. Dick, and Hidenori Tanaka, *A percolation model of emergence: Analyzing transformers trained on a formal language*, [arXiv:2408.12578](https://arxiv.org/abs/2408.12578).
- [16] Maya Okawa, Ekdeep S. Lubana, Robert Dick, and Hidenori Tanaka, *Compositional abilities emerge multiplicatively: Exploring diffusion models on a synthetic task*, *Adv. Neural Inf. Process. Syst.* **36**, 50173 (2023).
- [17] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler *et al.*, *Emergent abilities of large language models*, [arXiv:2206.07682](https://arxiv.org/abs/2206.07682).
- [18] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark *et al.*, *Training compute-optimal large language models*, *Adv. Neural Inf. Process. Syst.* **35**, 30016 (2022).
- [19] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra, *Grokking: Generalization beyond overfitting on small algorithmic datasets*, [arXiv:2201.02177](https://arxiv.org/abs/2201.02177).
- [20] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio, *On the difficulty of training recurrent neural networks*, in *International Conference on Machine Learning* (PMLR, 2013), pp. 1310–1318, <https://dl.acm.org/doi/10.5555/3042817.3043083>.
- [21] Kenji Doya *et al.*, *Bifurcations in the learning of recurrent neural networks 3*, *Learning (RTRL)* **3**, 17 (1992).
- [22] Ian J. Goodfellow, Oriol Vinyals, and Andrew M. Saxe, *Qualitatively characterizing neural network optimization problems*, [arXiv:1412.6544](https://arxiv.org/abs/1412.6544).
- [23] Gautam Reddy, *The mechanistic basis of data dependence and abrupt learning in an in-context classification task*, in *The Twelfth International Conference on Learning Representations* (ICLR, Vienna, Austria, 2023).
- [24] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen *et al.*, *In-context learning and induction heads*, [arXiv:2209.11895](https://arxiv.org/abs/2209.11895).
- [25] David Sussillo and Larry F. Abbott, *Generating coherent patterns of activity from chaotic neural networks*, *Neuron* **63**, 544 (2009).
- [26] Florian Hess, Zahra Monfared, Manuel Brenner, and Daniel Durstewitz, *Generalized teacher forcing for learning chaotic dynamics*, in *Proceedings of the 40th International Conference on Machine Learning*, ICML'23 (2023), <https://dl.acm.org/doi/10.5555/3618408.3618937>.
- [27] Aitor Lewkowycz, Yasaman Bahri, Ethan Dyer, Jascha Sohl-Dickstein, and Guy Gur-Ari, *The large learning rate phase of deep learning: The catapult mechanism*, [arXiv:2003.02218](https://arxiv.org/abs/2003.02218).
- [28] Yasaman Bahri, Jonathan Kadmon, Jeffrey Pennington, Sam S. Schoenholz, Jascha Sohl-Dickstein, and Surya Ganguli, *Statistical mechanics of deep learning*, *Annu. Rev. Condens. Matter Phys.* **11**, 501 (2020).
- [29] Sara Kangaslahti, Elan Rosenfeld, and Naomi Saphra, *Loss in the crowd: Hidden breakthroughs in language model training*, in *ICML 2024 Workshop on Mechanistic Interpretability* (ICML, Vienna, Austria, 2024).
- [30] Steven H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering* (CRC Press, Boca Raton, 2018).
- [31] Anton Maximilian Schäfer and Hans Georg Zimmermann, *Recurrent neural networks are universal approximators*, in *International Conference on Artificial Neural Networks* (Springer, New York, 2006), pp. 632–640, [10.1007/11840817\\_66](https://doi.org/10.1007/11840817_66).
- [32] Jack Carr, *Applications of Centre Manifold Theory* (Springer Science & Business Media, New York, 2012), Vol. 35.
- [33] Bifurcations are formally defined as shifts in system behavior, which alter the number or stability of fixed points, periodic orbits, or other invariant sets [26,30,34–39].
- [34] Dominik Schmidt, Georgia Koppe, Zahra Monfared, Max Beutelspacher, and Daniel Durstewitz, *Identifying nonlinear dynamical systems with multiple time scales and long-range dependencies*, in *International Conference on Learning Representations* (2021).
- [35] Robert Haschke and Jochen J. Steil, *Input space bifurcation manifolds of recurrent neural networks*, *Neurocomputing; Variable Star Bulletin* **64**, 25 (2005).
- [36] Alexander Rehmer and Andreas Kroll, *The effect of the forget gate on bifurcation boundaries and dynamics in recurrent neural networks and its implications for gradient-based optimization*, in *2022 International Joint Conference on Neural Networks (IJCNN)* (IEEE, Padua, Italy, 2022), pp. 01–08, [10.1109/IJCNN55064.2022.9892458](https://doi.org/10.1109/IJCNN55064.2022.9892458).
- [37] Viktor Avrutin, Michael Schanz, and Soumitro Banerjee, *Occurrence of multiple attractor bifurcations in the two-dimensional piecewise linear normal form map*, *Nonlinear Dyn.* **67**, 293 (2012).
- [38] A. Ganguli and S. Banerjee, *Dangerous bifurcation at border collision: When does it occur?*, *Phys. Rev. E* **71**, 057202 (2005).
- [39] Z. Monfared and D. Durstewitz, *Existence of n-cycles and border-collision bifurcations in piecewise-linear*

- continuous maps with applications to recurrent neural networks, *Nonlinear Dyn.* **101**, 1037 (2020).
- [40] Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Learning long-term dependencies with gradient descent is difficult*, *IEEE Trans. Neural Networks* **5**, 157 (1994).
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, *Attention is all you need*, *Adv. Neural Inf. Process. Syst.* **30**, 6000 (2017), <https://dl.acm.org/doi/10.5555/3295222.3295349>.
- [42] As a side note for precision, for our derivations in this section, we assume that  $f[x(t)]$  is a sufficiently smooth vector field that can be Taylor expanded to at least second order, with a nonvanishing quadratic coefficient along the slow-mode direction. The exact assumption will be clear at the end of the calculation.
- [43] Yuri A. Kuznetsov, *Elements of Applied Bifurcation Theory* (Springer, New York, 1998).
- [44] To accommodate slow-point manifolds and not just isolated points, one could rework this definition. For our purposes, the distinction is not relevant for the canonical model.
- [45] David Sussillo and Omri Barak, *Opening the black box: Low-dimensional dynamics in high-dimensional recurrent neural networks*, *Neural Comput.* **25**, 626 (2013).
- [46] S. H. Strogatz and R. M. Westervelt, *Predicted power laws for delayed switching of charge-density waves*, *Phys. Rev. B* **40**, 10501 (1989).
- [47] D. Koch, A. Nandan, G. Ramesan, I. Tyukin, A. Gorban, and A. Koseska, *Ghost channels and ghost cycles guiding long transients in dynamical systems*, *Phys. Rev. Lett.* **133**, 047202 (2024).
- [48] Diederik P. Kingma and Jimmy Ba, *Adam: A method for stochastic optimization*, [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [49] Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern, Nati Srebro, and Benjamin Recht, *The marginal value of adaptive gradient methods in machine learning*, *Adv. Neural Inf. Process. Syst.* **30**, 4151 (2017), <https://dl.acm.org/doi/10.5555/3294996.3295170>.
- [50] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein, *Visualizing the loss landscape of neural nets*, *Adv. Neural Inf. Process. Syst.* **31** (2018), <https://dl.acm.org/doi/10.5555/3327345.3327535>.
- [51] Adrian Valente, Jonathan W. Pillow, and Srdjan Ostojic, *Extracting computational mechanisms from neural data using low-rank rnns*, *Adv. Neural Inf. Process. Syst.* **35**, 24072 (2022).
- [52] Alexis Dubreuil, Adrian Valente, Manuel Beiran, Francesca Mastrogiuseppe, and Srdjan Ostojic, *The role of population structure in computations through neural dynamics*, *Nat. Neurosci.* **25**, 783 (2022).
- [53] Fatih Dinc, Marta Blanco-Pozo, David Klindt, Francisco Acosta, Yiqi Jiang, Sadegh Ebrahimi, Adam Shai, Hidenori Tanaka, Peng Yuan, Mark J. Schnitzer, and Nina Miolane, *Latent computing by biological neural networks: A dynamical systems framework*, [arXiv:2502.14337](https://arxiv.org/abs/2502.14337).
- [54] Manuel Beiran, Alexis Dubreuil, Adrian Valente, Francesca Mastrogiuseppe, and Srdjan Ostojic, *Shaping dynamics with multiple populations in low-rank recurrent networks*, *Neural Comput.* **33**, 1572 (2021).
- [55] Francesca Mastrogiuseppe and Srdjan Ostojic, *Linking connectivity, dynamics, and computations in low-rank recurrent neural networks*, *Neuron* **99**, 609 (2018).
- [56] F. Dinc, Code for “Ghost Mechanism: An Analytical Model of Abrupt Learning in Recurrent Networks”. Zenodo, (2024), [10.5281/zenodo.13689162](https://zenodo.org/record/13689162).
- [57] A. J. Bray and D. S. Dean, *Statistics of critical points of Gaussian fields on large-dimensional spaces*, *Phys. Rev. Lett.* **98**, 150201 (2007).
- [58] Nicolas Y. Masse, Guangyu R. Yang, H. Francis Song, Xiao-Jing Wang, and David J. Freedman, *Circuit mechanisms for the maintenance and manipulation of information in working memory*, *Nat. Neurosci.* **22**, 1159 (2019).
- [59] Guangyu Robert Yang, Madhura R. Joglekar, H. Francis Song, William T. Newsome, and Xiao-Jing Wang, *Task representations in neural networks trained to perform many cognitive tasks*, *Nat. Neurosci.* **22**, 297 (2019).
- [60] Niru Maheswaranathan, Alex Williams, Matthew Golub, Surya Ganguli, and David Sussillo, *Reverse engineering recurrent networks for sentiment classification reveals line attractor dynamics*, *Adv. Neural Inf. Process. Syst.* **32**, 15696 (2019), <https://dl.acm.org/doi/10.5555/3454287.3455693>.
- [61] Antônio H. Ribeiro, Koen Tiels, Luis A. Aguirre, and Thomas Schön, *Beyond exploding and vanishing gradients: Analysing rnn training using attractors and smoothness*, in *International Conference on Artificial Intelligence and Statistics, Palermo, Sicily, Italy*. (PMLR, 2020), pp. 2370–2380, <https://proceedings.mlr.press/v108/ribeiro20a.html>.
- [62] Niru Maheswaranathan, Alex Williams, Matthew Golub, Surya Ganguli, and David Sussillo, *Universality and individuality in neural dynamics across large populations of recurrent networks*, *Adv. Neural Inf. Process. Syst.* **32**, 15629 (2019), <https://dl.acm.org/doi/10.5555/3454287.3455687>.
- [63] Xavier Glorot and Yoshua Bengio, *Understanding the difficulty of training deep feedforward neural networks*, in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, Sardinia, Italy, JMLR Workshop and Conference Proceedings* (PMLR, 2010), pp. 249–256, <https://proceedings.mlr.press/v9/glorot10a.html>.
- [64] Xavier Glorot, Antoine Bordes, and Yoshua Bengio, *Deep sparse rectifier neural networks*, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, JMLR Workshop and Conference Proceedings* (PMLR, 2011), pp. 315–323, <https://proceedings.mlr.press/v15/glorot11a.html>.
- [65] That is, unless initialization occurs in a parameter regime, which in full-rank RNNs leads to chaos [66]. Studying an equivalent initialization regime in low-rank RNNs constitutes an entirely different research direction.
- [66] H. Sompolsky, A. Crisanti, and H. J. Sommers, *Chaos in random neural networks*, *Phys. Rev. Lett.* **61**, 259 (1988).
- [67] Blake A. Richards, Timothy P. Lillicrap, Philippe Beaudoin, Yoshua Bengio, Rafal Bogacz, Amelia Christensen, Claudia Clopath, Rui Ponte Costa, Archy de Berker, Surya Ganguli

- et al.*, *A deep learning framework for neuroscience*, *Nat. Neurosci.* **22**, 1761 (2019).
- [68] Valerio Mante, David Sussillo, Krishna V. Shenoy, and William T. Newsome, *Context-dependent computation by recurrent dynamics in prefrontal cortex*, *Nature (London)* **503**, 78 (2013).
- [69] Matthew G. Perich, Charlotte Arlt, Sofia Soares, Megan E. Young, Clayton P. Mosher, Juri Minxha, Eugene Carter, Ueli Rutishauser, Peter H. Rudebeck, Christopher D. Harvey *et al.*, *Inferring brain-wide interactions using data-constrained recurrent neural network models*, *bioRxiv* (2021), pp. 2020–12, [10.1101/2020.12.18.423348](https://doi.org/10.1101/2020.12.18.423348).
- [70] Matthew G. Perich, Devika Narain, and Juan A. Gallego, *A neural manifold view of the brain*, *Nat. Neurosci.* **28**, 1582 (2025).
- [71] Matthew G. Perich and Kanaka Rajan, *Rethinking brain-wide interactions through multi-region ‘network of networks’ models*, *Current Opinion Neurobiol.* **65**, 146 (2020).
- [72] Mengyu Liu, Aditya Nair, Nestor Coria, Scott W. Linderman, and David J. Anderson, *Encoding of female mating dynamics by a hypothalamic line attractor*, *Nature (London)* **634**, 901 (2024).
- [73] Abhranil Das and Ila R. Fiete, *Systematic errors in connectivity inferred from activity in strongly recurrent networks*, *Nat. Neurosci.* **23**, 1286 (2020).
- [74] Kiyohito Iigaya, Madalena S. Fonseca, Masayoshi Murakami, Zachary F. Mainen, and Peter Dayan, *An effect of serotonergic stimulation on learning rates for rewards apparent after long intertrial intervals*, *Nat. Commun.* **9**, 2477 (2018).
- [75] Cooper D. Grossman, Bilal A. Bari, and Jeremiah Y. Cohen, *Serotonin neurons modulate learning rate through uncertainty*, *Curr. Biol.* **32**, 586 (2022).
- [76] Luke T. Coddington, Sarah E. Lindo, and Joshua T. Dudman, *Mesolimbic dopamine adapts the rate of learning from action*, *Nature (London)* **614**, 294 (2023).
- [77] Kanaka Rajan, Christopher D. Harvey, and David W. Tank, *Recurrent network models of sequence generation and memory*, *Neuron* **90**, 128 (2016).
- [78] Joaquin M. Fuster and Garrett E. Alexander, *Neuron activity related to short-term memory*, *Science* **173**, 652 (1971).
- [79] Saurabh Vyas, Matthew D. Golub, David Sussillo, and Krishna V. Shenoy, *Computation through neural population dynamics*, *Annu. Rev. Neurosci.* **43**, 249 (2020).
- [80] Christopher J. Honey, Thomas Thesen, Tobias H. Donner, Lauren J. Silbert, Chad E. Carlson, Orrin Devinsky, Werner K. Doyle, Nava Rubin, David J. Heeger, and Uri Hasson, *Slow cortical dynamics and the accumulation of information over long timescales*, *Neuron* **76**, 423 (2012).
- [81] Fatih Dinc, Adam Shai, Mark Schnitzer, and Hidenori Tanaka, *Cornn: Convex optimization of recurrent neural networks for rapid inference of neural dynamics*, *Adv. Neural Inf. Process. Syst.* **36**, 51273 (2023).
- [82] Ábel Ságoti, Guillermo Martín-Sánchez, Piotr Sokol, and Memming Park, *Back to the continuous attractor*, *Adv. Neural Inf. Process. Syst.* **37**, 66856 (2024).
- [83] Bariscan Kurtkaya, Fatih Dinc, Mert Yuksekgonul, Marta Blanco-Pozo, Ege Cirakman, Mark J. Schnitzer, Yucel Yemez, Hidenori Tanaka, Peng Yuan, and Nina Miolane, *Dynamical phases of short-term memory mechanisms in RNNs*, in *Proceedings of the 42nd International Conference on Machine Learning, ICML’25 (2025)*, <https://dl.acm.org/doi/10.5555/3780338.3781597>.
- [84] Renate Krause, Matthew Cook, Sepp Kollmorgen, Valerio Mante, and Giacomo Indiveri, *Operative dimensions in unconstrained connectivity of recurrent neural networks*, *Adv. Neural Inf. Process. Syst.* **35**, 17073 (2022).
- [85] Ann Huang, Satpreet H. Singh, Flavio Martinelli, and Kanaka Rajan, *Measuring and controlling solution degeneracy across task-trained recurrent neural networks*, *arXiv:2410.03972*.
- [86] Friedrich Schuessler, Francesca Mastrogiuseppe, Alexis Dubreuil, Srdjan Ostojic, and Omri Barak, *The interplay between randomness and structure during learning in RNNs*, *Adv. Neural Inf. Process. Syst.* **33**, 13352 (2020), <https://dl.acm.org/doi/abs/10.5555/3495724.3496844>.
- [87] Laura Balzano, Tianjiao Ding, Benjamin D. Haeffele, Soo Min Kwon, Qing Qu, Peng Wang, Zhangyang Wang, and Can Yaras, *An overview of low-rank structures in the training and adaptation of large models*, *arXiv:2503.19859*.
- [88] Albert Gu and Tri Dao, *Mamba: Linear-time sequence modeling with selective state spaces*, in *First Conference on Language Modeling, Philadelphia, PA, USA (COLM, 2024)*.
- [89] Christian Klos and Raoul-Martin Memmesheimer, *Smooth exact gradient descent learning in spiking neural networks*, *Phys. Rev. Lett.* **134**, 027301 (2025).
- [90] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass, *A solution to the learning dilemma for recurrent networks of spiking neurons*, *Nat. Commun.* **11**, 3625 (2020).
- [91] Michael Pfeiffer and Thomas Pfeil, *Deep learning with spiking neurons: Opportunities and challenges*, *Front. Neurosci.* **12**, 409662 (2018).
- [92] Zhaoyang Niu, Guoqiang Zhong, and Hui Yu, *A review on the attention mechanism of deep learning*, *Neurocomputing; Variable Star Bulletin* **452**, 48 (2021).
- [93] Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna, *Universally sloppy parameter sensitivities in systems biology models*, *PLoS Comput. Biol.* **3**, e189 (2007).
- [94] Scott Kirkpatrick, C. Daniel Gelatt Jr, and Mario P. Vecchi, *Optimization by simulated annealing*, *Science* **220**, 671 (1983).
- [95] Neal A. Hessler and Allison J. Doupe, *Social context modulates singing-related neural activity in the songbird forebrain*, *Nat. Neurosci.* **2**, 209 (1999).
- [96] Mimi H. Kao, Brian D. Wright, and Allison J. Doupe, *Neurons in a forebrain nucleus required for vocal plasticity rapidly switch between precise firing and variable bursting depending on social context*, *J. Neurosci.* **28**, 13232 (2008).